

School of Electronic Engineering
and Computer Science

Final Report

Programme of study:
Computer Science BSc

Project Title:
SkillFindr

Supervisor:
Aisha Abuelmatti

Student Name:
Mathis Weil

Date: 6th May 2025

Final Year
Undergraduate Project 2024/25

Abstract

IBM SkillsBuild is a free educational platform offering a broad range of technical and professional courses. However, users often struggle to identify relevant content due to the platform's breadth and lack of intelligent search capabilities. *SkillFindr* is an AI-enabled learning assistant developed in collaboration with IBM to address this challenge by combining a natural language interface, semantic search, and retrieval-augmented generation (RAG). This project specifically focuses on designing and implementing the core semantic search engine that powers SkillFindr's intelligent retrieval capabilities.

The system comprises three main components: (1) a dynamic web crawler to extract structured and unstructured metadata from over 1,200 courses; (2) a hybrid database design combining schema-based and vector-based storage using PostgreSQL and pgvector; and (3) a semantic search engine built with Sentence-BERT embeddings to enable natural language querying over the indexed courses.

Evaluation was conducted using a manually annotated dataset of 50 user-like queries and standard information retrieval metrics (Precision@K, Recall@K, MRR, MAP@K, nDCG@K). The semantic system significantly outperformed keyword and BM25 baselines, achieving a Recall@20 of 0.899, MRR of 0.843, and 100% query coverage, while also demonstrating lower latency and higher retrieval relevance. The system was deployed using FastAPI to expose modular endpoints for semantic retrieval and future integration into conversational agents.

These findings validate the use of embedding-based semantic search to address user intent and improve content discoverability on educational platforms. While SkillFindr encompasses broader capabilities, this project establishes the foundational semantic retrieval layer and outlines future directions such as multilingual indexing, domain-specific embedding fine-tuning, and integration with RAG pipelines for adaptive, learner-centric course discovery.

C contents

Chapter 1:	<i>Introduction</i>	5
1.1	Background	5
1.2	Problem Statement	5
1.3	Aim	6
1.4	Objectives	6
1.5	Research Questions	7
1.6	Report Structure	8
Chapter 2:	<i>Literature Review</i>	9
2.1	Semantic Search: Definition and Importance	9
2.2	Embedding Models for Semantic Search	9
2.3	Web Crawling IBM SkillsBuild	10
2.4	Database Design: Schema-Based vs. Vector Based Systems	11
2.5	Gaps in Current Research	11
2.6	Conclusion	12
Chapter 3:	<i>Methodology</i>	14
3.1	Web Crawling and Data extraction	14
3.2	Data Preprocessing and Cleaning	17
3.3	Hybrid Database Design	19
3.4	Embedding Strategy	20
3.5	Semantic Search Design	21
Chapter 4:	<i>Implementation</i>	23
4.1	Web Scraping and HTML Parsing	23
4.2	Data Cleaning and Normalisation	23
4.3	Embedding Generation	24
4.4	Database Storage	24
4.5	Query Execution and Retrieval	25
4.6	Configuration and Utilities	26
4.7	API Deployment	27
Chapter 5:	<i>Evaluation</i>	28
5.1	Testing Methods	28
5.2	Results	31
5.3	Discussion and Research Question Analysis	36

5.4	Summary	38
Chapter 6: Conclusion		39
6.1	Overview	39
6.2	Summary of Achievements	39
6.3	Reflection on Research Questions	40
6.4	Limitations	40
6.5	Future Work	41
References		42
Appendix A – Risk Assessment		45
Appendix B – Project Plan		47
Appendix C – Key Implementation code		48
Appendix D - Evaluation Code and Metrics		57
Appendix E - Example Evaluation Queries		62

Chapter 1: Introduction

1.1 Background

IBM SkillsBuild is a free educational platform offering a wide range of technical and professional courses. These courses cater to all proficiency levels, from beginners to advanced learners, and cover diverse topics such as web development, artificial intelligence, data science, and cybersecurity (IBM SkillsBuild Course Catalogue, n.d.). Credentials earned through SkillsBuild courses are recognised across the industry, making them a valuable resource for showcasing skills and advancing careers (IBM SkillsBuild Digital Credentials, n.d.). Despite its benefits, users often face challenges in finding the right courses due to the extensive array of choices and varying levels of proficiency. They may struggle to identify courses suitable for their skill level, hesitate over course relevance, or feel unsure about which learning paths align with their goals. These challenges impact the user experience and limit the platform's accessibility. To address these issues, this project focuses on developing a semantic search engine to enhance course discoverability. It involves crawling the IBM SkillsBuild platform to gather metadata, designing a hybrid database system (schema-based and vector-based) for efficient storage and retrieval, and implementing a semantic search engine powered by embedding models. This work contributes the semantic retrieval component of *SkillFindr*, a broader IBM-led initiative to build an AI-enabled learning assistant that integrates a user-facing interface, Retrieval-Augmented Generation (RAG), and large language models (LLMs) for personalised educational support.

1.2 Problem Statement

The IBM SkillsBuild platform currently relies on keyword-based search mechanisms that often fails to meet user expectations. With over a thousand courses available across a variety of technical and professional topics (IBM SkillsBuild, n.d.), users usually struggle to find content tailored to their skill level, interests, or career goals. This lack of precision arises from:

1. **Ineffective query understanding:** Keyword-based search lacks the ability to interpret user intent or contextual nuances.
2. **Limited Scalability:** As the platform's catalogue continues to grow, manually browsing or filtering through courses becomes increasingly overwhelming.
3. **Impact on User Experience:** Frustration with poor search results discourages user engagement and undermines the platform's usability.

These challenges highlight the need for a semantic search engine that can process natural language queries, understand user intent, and deliver accurate, personalised results while leveraging the context of the discussion. By addressing this gap, the project aims to significantly improve the user experience and accessibility of IBM SkillsBuild.

1.3 Aim

The aim of this project is to enhance the user experience and accessibility of the IBM SkillsBuild platform by developing the semantic search engine that underpins an AI-powered learning assistant, *SkillFindr*. This component will increase the discoverability of courses using advanced web crawling, hybrid database design, and state-of-the-art embedding models. The semantic search engine will process natural language queries, interpret user intent and deliver tailored, contextually relevant course recommendations. This system will overcome and address the limitations of the traditional keyword-based search methods by giving a scalable and user-friendly way to look through the extensive IBM SkillsBuild course catalogue.

1.4 Objectives

To achieve the aim of this project, the following objectives will be addressed:

1. Data Collection through Web Crawling:
 - a. Develop and implement a web crawler using tools like Python, Selenium and BeautifulSoup4 to collect structured and unstructured metadata from IBM SkillsBuild course catalogue.
 - b. Parse and clean the collected data using techniques such as HTML tag removal, Regex pattern matching, and Unicode normalisation. This step also includes extracting meaningful metadata such as course durations, available languages and star ratings, while removing boilerplate text to ensure high-quality input for semantic analysis.
2. Database Design and Implementation:
 - a. Design a hybrid database system combining schema-based and vector-based approaches:
 - i. Schema-based database for structured metadata such as course titles, languages, and duration.
 - ii. Vector database for storing embeddings representing course descriptions and other unstructured content.
 - b. Ensure the database supports efficient data retrieval and integration with the semantic search engine.
3. Embedding Model Analysis and Selection:
 - a. Research and evaluate pre-trained embedding models such as Sentence-BERT and text-embedding-ada-002.
 - b. Perform qualitative and quantitative tests to determine the best model for handling natural language queries and matching them with relevant courses.
4. Semantic Search Engine Development:
 - a. Implement a semantic search engine that processes user queries in natural language, interprets intent, and retrieves accurate and personalised course recommendations.
 - b. Integrate the search engine with the hybrid database for real-time query responses.
5. System Evaluation and Iteration:

- a. Conduct extensive testing of the semantic search engine using sample queries to evaluate performance, accuracy, and user relevance. This will include comparative analysis against a traditional keyword-based search baseline to assess improvements in retrieval quality, contextual matching, and user satisfaction.
 - b. Document evaluation-driven insights and identify potential improvements to support future system scalability and usability.
6. API Deployment and RAG Integration:
 - a. Develop a FastAPI-based RESTful API to expose the semantic search engine and support real-time access to course recommendations.
 - b. Define modular endpoints for semantic retrieval, enabling future integration with RAG, though RAG implementation itself is beyond the scope of this project.
 - c. Implement input validation, filtering, and context-aware response structures using SQLAlchemy and .env-driven configuration.

A full project timeline, including milestones and resource allocation, is provided in *Appendix B (Project Plan)*.

1.5 Research Questions

1.5.1 Research Question 1

How can semantic search enhance course discoverability on the IBM SkillsBuild platform?

- **Sub-questions:**
 1. What are the limitations of current keyword-based search systems on IBM SkillsBuild?
 2. How do embedding models like Sentence-BERT and text-embedding-ada-002 improve query intent understanding?
 3. What are the challenges of implementing a domain-specific semantic search for educational platforms?

1.5.2 Research Question 2

How can web crawling strategies be optimised for reliable and scalable metadata extraction in educational platforms like IBM SkillsBuild?

- **Sub-questions:**
 1. What are the technical and ethical challenges in crawling dynamically rendered educational platforms?
 2. How do different tools (e.g. Selenium, Scrapy, Puppeteer) compare in extracting structured and unstructured metadata?
 3. What preprocessing strategies are essential to transform scraped content into high-quality semantic input?

1.5.3 Research Question 3

How can hybrid database systems integrate structured and unstructured data for effective semantic search?

- **Sub-questions:**
 1. What are the benefits and limitations of using hybrid relational–vector databases for educational metadata retrieval?
 2. How can structured metadata and semantic embeddings be effectively integrated within a unified search architecture?

1.6 Report Structure

This report is organised as follows:

- **Chapter 1: Introduction**

This chapter introduces the background, problem statement, aim, objectives, and research questions of the project. It also outlines the structure of the report.
- **Chapter 2: Literature Review**

A critical review of relevant literature, including semantic search, embedding models, web crawling, and database design. This chapter identifies gaps in current research and establishes the foundation for the project's methodology.
- **Chapter 3: Methodology**

This chapter details the approach used to achieve the project objectives. It covers web crawling techniques, database system design, embedding model evaluation, and the development of the semantic search engine.
- **Chapter 4: Implementation**

A comprehensive explanation of the system's implementation, including web crawling, database integration, embedding model selection, semantic search engine development and FastAPI-based RESTful API deployment.
- **Chapter 5: Evaluation**

This chapter describes the testing and evaluation methods, presents results on the performance and usability of the semantic search engine, and compares it to traditional keyword-based search systems.
- **Chapter 6: Conclusion and Future Work**

The final chapter summarises the findings, discusses the implications, and outlines potential future developments to enhance the system further.
- **References**

A list of all sources cited in the report, adhering to proper academic referencing standards.
- **Appendices**

Supplementary materials, such as code snippets, additional data, and experimental results, are included for reference.

Chapter 2: Literature Review

2.1 Semantic Search: Definition and Importance

Semantic search enables retrieval based on contextual meaning rather than exact keyword matches. Unlike traditional keyword-based approaches, which rely on lexical overlap, semantic systems use embeddings generated by machine learning models to capture relationships between words and interpret user intent. This leads to more accurate and user-aligned results, especially in domains with diverse and complex query formulations such as education.

Traditional information retrieval models, as outlined by Manning et al. (2008), laid the groundwork for query formulation and ranking but struggle with ambiguity and synonymy. These limitations have been addressed by transformer-based architectures like BERT (Vaswani et al., 2017), which power embedding models such as Sentence-BERT (Reimers & Gurevych, 2019) and OpenAI's Ada-002 (OpenAI, 2023). These models allow natural language queries and documents to be embedded in a shared vector space, enabling retrieval based on cosine similarity.

While semantic search has demonstrated success in commercial domains including e-commerce and support systems (Muennighoff, 2022), its application in educational platforms remains limited. Educational queries often blend instructional intent with constraints like duration, level, and certification goals. Therefore, systems must interpret both unstructured descriptions and structured metadata.

To address these challenges, this project implements a semantic search engine that embeds course content and user queries using Sentence-BERT and retrieves matches using cosine similarity via pgvector in PostgreSQL. While Retrieval-Augmented Generation (RAG) approaches (Lewis et al., 2020) offer promising ways to extend semantic retrieval with generative LLMs, such integration lies beyond the scope of this work. However, the modular FastAPI interface developed in this project is designed to support such extensions, enabling future integration with conversational systems that require semantically relevant context retrieval.

2.2 Embedding Models for Semantic Search

Embedding models encode textual data into high-dimensional vectors, allowing for context-aware semantic similarity comparisons. Unlike keyword-based methods like BM25 (Robertson & Zaragoza, 2009), which rely on term frequency and lexical overlap, embeddings represent concepts in a continuous space, enabling the retrieval of semantically relevant documents even when no keywords overlap.

Transformer-based models such as BERT (Devlin et al., 2019) introduced contextual embeddings but are computationally intensive. Sentence-BERT (Reimers & Gurevych, 2019), used in this project, adapts BERT for efficient

sentence-level similarity by fine-tuning it with siamese and triplet network architectures. It produces 384-dimensional embeddings suitable for real-time retrieval, making it well-suited for semantic search on educational datasets.

To represent course content semantically, this project constructs input strings from course titles, descriptions, and tags, which are embedded using all-MiniLM-L6-v2 from SentenceTransformers. This model was chosen based on a balance of efficiency, accuracy, and its proven performance in retrieval benchmarks such as BEIR (Thakur et al., 2021) and MTEB (Muennighoff et al., 2022). The resulting embeddings are stored in PostgreSQL with the pgvector extension and used for cosine similarity comparisons with embedded queries.

Recent models like OpenAI's text-embedding-ada-002 and SGPT (Muennighoff, 2022) demonstrate advancements in embedding architectures, particularly for Retrieval-Augmented Generation (RAG) pipelines. However, these models were not adopted in this project due to API restrictions, cost, or architectural complexity. Still, their strong performance underscores the value of embedding-based search for educational systems.

While fine-tuning embedding models on domain-specific datasets like cybersecurity, data-science or course metadata could improve semantic alignment (Nguyen et al., 2022), it was not pursued due to the lack of sufficient labelled data and computational constraints. The current system performs well in a general-purpose setting but could benefit from future domain-specific adaptation.

For similarity computation, cosine similarity was used, as it provides a scale-invariant distance metric suitable for comparing dense vectors. Although more nuanced approaches such as TF-PIV or probabilistic scoring exist (Roelleke, 2013), they were not implemented, as the goal of this project was to validate the performance of a baseline semantic system using widely adopted methods.

In summary, embedding models like Sentence-BERT provide a scalable and effective foundation for semantic search in educational platforms. Their integration into this system, alongside pgvector and modular FastAPI endpoints, supports accurate, low-latency query processing and sets the stage for future enhancements such as fine-tuning, multilingual embeddings, or RAG-based dialogue agents.

2.3 Web Crawling IBM SkillsBuild

To enable semantic retrieval and course discovery on IBM SkillsBuild, a custom web crawler was developed to extract both structured metadata (e.g., course titles, durations, tags, and ratings) and unstructured content (e.g., descriptions and learning objectives). This data populates a hybrid storage architecture that supports full-text and embedding-based search.

The crawler uses Selenium to interact with dynamically rendered content and BeautifulSoup for parsing HTML (Crummy, 2020; Selenium Docs, 2024). This combination allows for reliable extraction of content that would be inaccessible via static scraping methods. Tab-switching, scroll simulation, and dynamic

content detection were implemented to capture all course records across different sections.

Challenges included inconsistent metadata formatting and dynamic page loads, which required robust preprocessing to standardise the output for schema-based and vector storage. Crawling was conducted ethically, in compliance with robots.txt directives and IBM's terms of service, in line with established best practices (Olston & Najork, 2010).

The extracted metadata forms the basis for indexing and embedding workflows described in Chapters 3 and 4. While the current system is focused on English-language content, future enhancements could include multilingual crawling and adaptive techniques that automatically adjust to UI changes (Khder, 2021).

2.4 Database Design: Schema-Based vs. Vector Based Systems

Integrating structured and unstructured data is essential for semantic search in educational systems, where both metadata and textual course content contribute to retrieval quality. Structured elements such as course titles, durations, and prerequisites are best managed with relational databases, which support efficient filtering and querying through defined schemas (Liu & Gawlick, 2015). In contrast, unstructured data, like course descriptions, requires semantic interpretation, which is enabled by vector databases that store high-dimensional embeddings and support similarity-based retrieval using metrics such as cosine distance and approximate nearest neighbour search (Johnson, Douze & Jégou, 2019; pgvector, 2023). This hybrid architecture enables flexible, high-performance access to diverse data types, which is crucial for educational discovery systems.

This project adopts a hybrid approach by integrating both storage paradigms in a unified PostgreSQL system extended with pgvector. Structured metadata is stored in traditional relational fields, while 384-dimensional vectors are indexed for efficient semantic comparison. This enables hybrid querying, where users can filter by duration or tags while retrieving semantically relevant results from descriptions.

Prior research supports the hybrid design adopted in this project. Knowledge-enriched frameworks that combine schema-based and vector-based representations have been shown to improve semantic alignment across heterogeneous datasets (Ma, Molnár & Tarcsi, 2022).

By combining both database paradigms, this architecture enables more flexible, scalable, and user-aligned retrieval, crucial for educational platforms like IBM SkillsBuild where search precision and semantic relevance must coexist

2.5 Gaps in Current Research

Despite advancements in semantic search, embedding models, and hybrid databases, challenges remain in adapting these technologies to specialised

educational platforms like IBM SkillsBuild, particularly in developing domain-specific embeddings, unifying structured and unstructured data, and ensuring diverse, intent-aligned retrieval.

2.5.1 Domain-Specific Embedding Models

Most embedding models, such as Sentence-BERT (Reimers & Gurevych, 2019) and Ada (OpenAI, 2023), are trained for general-purpose tasks. While effective in broad domains (Thakur et al., 2021), they struggle to capture the nuanced semantics required in specialised educational contexts. Domain-tuned models like SciBERT (Beltagy et al., 2019) and BioBERT (Lee et al., 2020) demonstrate the value of specialisation, yet equivalent models for educational platforms remain lacking. Although fine-tuning offers a solution (Nguyen et al., 2022), it is constrained by limited annotated datasets and high resource demands.

2.5.2 Hybrid Database Integration

Seamlessly combining schema-based and vector-based databases is underexplored in education-focused search systems. While pgvector and platforms like Pinecone support vector storage (Maryamah et al., 2024), effective integration with relational queries and filters (e.g. by duration or course type) remains an open engineering challenge. Kraska et al. (2018) discuss optimising learned index structures, but few studies address hybrid architectures needed for complex educational metadata.

2.5.3 Retrieval Optimisation for Educational Platforms

Maximising both semantic relevance and result diversity is critical but rarely addressed in educational retrieval systems. Methods like Maximum Marginal Relevance (MMR) (Carbonell & Goldstein, 1998) help balance relevance and redundancy, yet their use in course search remains limited. Maryamah et al. (2024) showed potential for MMR in academic datasets but emphasised the need for adaptive parameter tuning, especially relevant for learners with varying goals (e.g. certification vs. skill-building).

2.5.4 Applications to Educational Contexts

Despite growing interest in embedding-based search, its adaptation to educational platforms, especially those targeting technical subjects, remains limited (Yoo et al., 2018). Most work focuses on chatbots or generic content retrieval, not the metadata-rich, goal-oriented nature of platforms like IBM SkillsBuild. Tailored semantic retrieval that incorporates both structured metadata (e.g., tags, durations) and contextual embeddings is necessary to align with learner needs.

2.6 Conclusion

This literature review explored key components of semantic search systems in the context of educational platforms, with a focus on embedding models, web crawling, and hybrid database design. While transformer-based embeddings such as Sentence-BERT and Ada-002 offer powerful tools for intent-based

retrieval, their domain adaptation to fields like cybersecurity or data science remains underexplored due to limited annotated data and high resource demands.

The review also highlighted the importance of metadata acquisition through ethical and robust web crawling practices. Tools like Selenium and BeautifulSoup are well-suited for extracting structured and unstructured educational content, though challenges persist with dynamic content rendering and multilingual data coverage.

Hybrid database architectures combining schema-based relational models with vector-based similarity search, such as PostgreSQL with pgvector, offer practical solutions for integrating structured filters with semantic relevance. However, seamless integration methods, especially in educational contexts, remain an open research challenge.

Finally, the review identifies core gaps in current literature: the lack of domain-specific embeddings, limited research on hybrid storage systems tailored to educational data, and insufficient exploration of retrieval diversity optimisation. These limitations underscore the need for further applied research to bridge technical solutions with the nuanced requirements of course discovery in platforms like IBM SkillsBuild.

Chapter 3: Methodology

3.1 Web Crawling and Data extraction

To build an effective semantic search engine, a high-quality dataset of course metadata from the IBM SkillsBuild platform is required. This metadata includes titles, course durations, descriptions, and other structured information. Since no direct API access is available, a web scraping pipeline was developed to automate the extraction of this content.

3.1.1 Web Crawling Strategy

Due to the lack of a public API, course data from IBM SkillsBuild is collected via web scraping using Python 3.12, Selenium WebDriver, and BeautifulSoup4 with the lxml parser. This stack was selected for its balance of dynamic interaction support, parsing speed, and code maintainability.

Selenium is used to simulate browser sessions, enabling the scraper to log in with secured credentials, navigate dynamic filters, and trigger asynchronous content loading (e.g., repeated “Show More” clicks). These capabilities are essential for scraping JavaScript-rendered content, as traditional crawlers fail to process dynamic DOM structures (Khder, 2021; Mitchell, 2018).

Once the content is rendered, BeautifulSoup4 parses the HTML to extract key metadata (title, tags, description, duration). It is lightweight, readable, and integrates seamlessly with Selenium for post-render parsing (Crummy, 2020).

To ensure high performance and resilience, lxml is used as the backend parser. Compared to html.parser and html5lib, lxml is faster, handles malformed HTML better, and processes deeply nested structures efficiently, traits critical for parsing complex educational content. Benchmark results confirm its superior execution speed and reliability when used with BeautifulSoup (Uzun et al., 2018).

Configuration parameters (e.g., course categories, language filters) are stored in a JSON file. A key setting enforces an English-language filter (&languages=en) at the URL level, aligning the dataset with the monolingual semantic embedding pipeline and reducing the need for downstream language detection or filtering.

3.1.2 Course Link extraction

The scraper iterates through course categories and search sections defined in the config.json file. Instead of interacting with the search input box, it directly constructs search URLs using a consistent structure such as:

```
https://skills.yourlearning.ibm.com/search/activities/q=cybersecurity&languages=en
```

In this URL structure, the segment /activities/ denote the section, q= defines the search keyword (category), and languages= specifies the language filter. By modifying these URL parameters, the scraper can systematically query different course categories and restrict results to English-language content.

This approach ensures alignment with the embedding model's capabilities while reducing downstream noise. The scraper then programmatically expands course listings by repeatedly clicking the "Show More" button until all content is visible. Duplicate links are avoided by maintaining a set of already-visited URLs. Valid links are then collected and queued for course-level scraping.

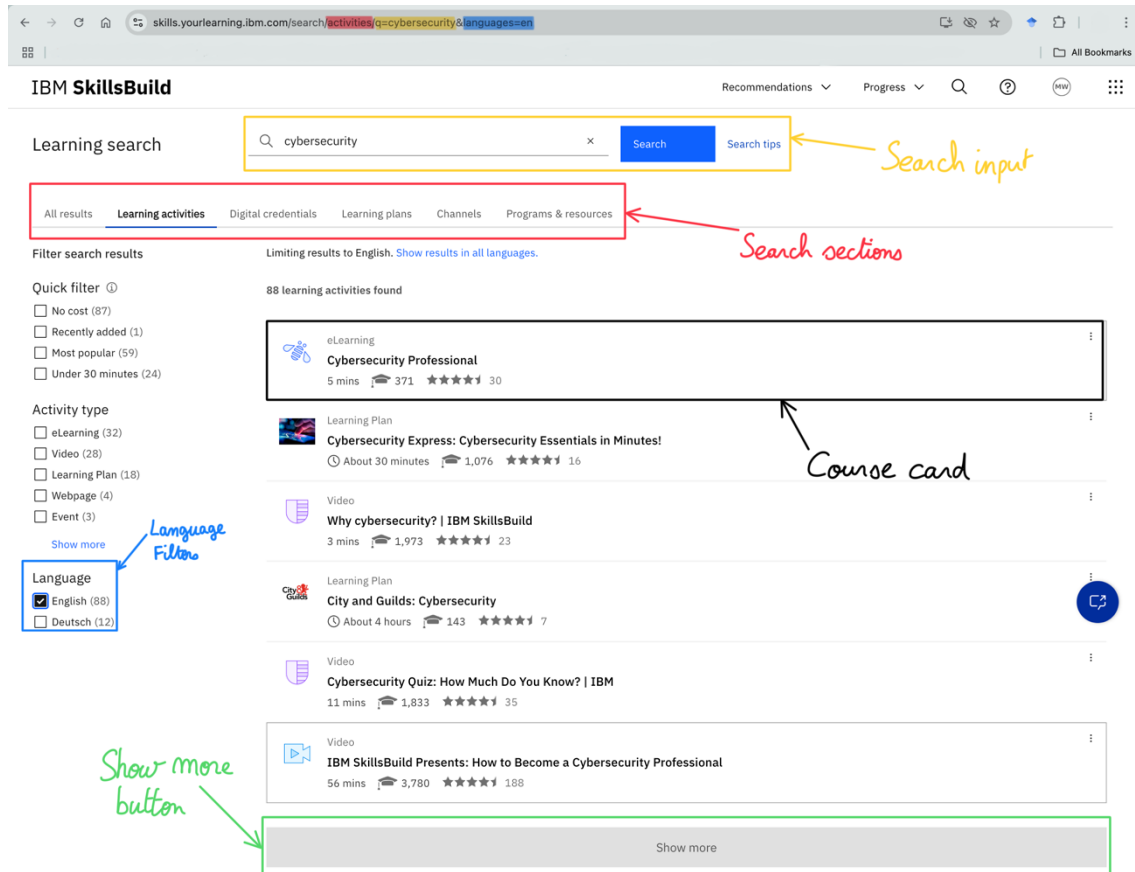


Figure 1. Figure: Annotated screenshot of the IBM SkillsBuild course search interface.

This figure illustrates the structure of the course search results page, highlighting key elements relevant to the scraping logic: (1) the search input and search sections used to generate the crawling URLs, (2) the language filter applied during crawling to restrict results to English courses, (3) the course card containers that encapsulate each learning activity, and (4) the "Show more" button, which dynamically loads additional results and must be programmatically clicked by the scraper.

3.1.3 Course Page Parsing

Upon accessing a course detail page, BeautifulSoup4 parses the HTML to extract key fields: URL, title, duration, rating, description, tags, and more. All extracted data is saved in both .csv and .json formats. The CSV format supports debugging and inspection, while the JSON format is used for structured downstream processing.

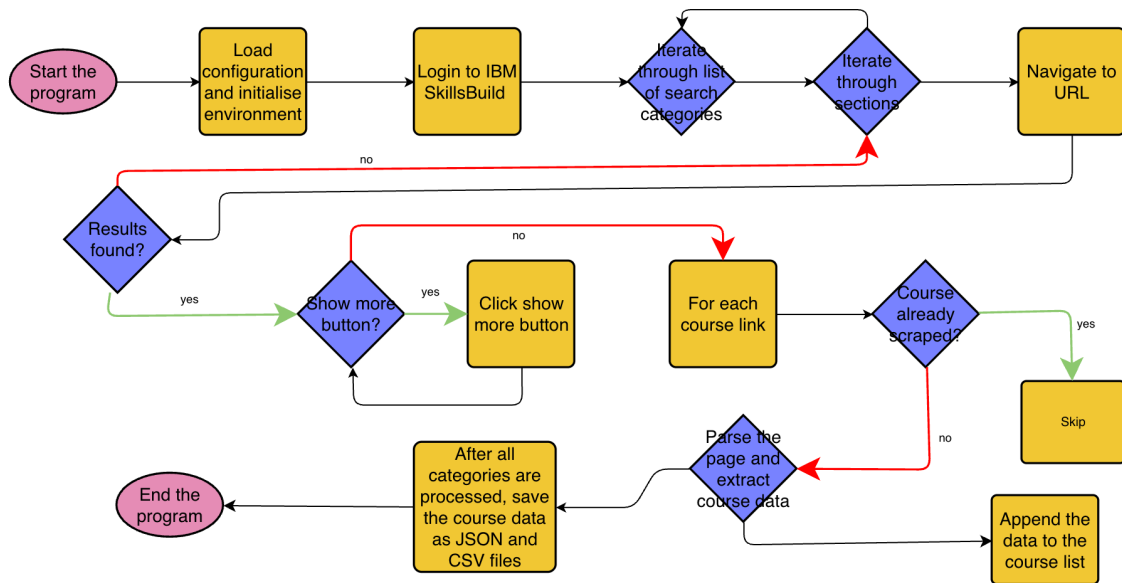


Figure 2. Overview of the IBM SkillsBuild scraping workflow.

The process begins with configuration and login, followed by iterating through categories and sections to construct search URLs. After handling dynamic content via the “Show more” button, the scraper extracts metadata from each course unless previously scraped. Final data is saved as JSON and CSV. The flow includes checks for duplicates and dynamic rendering to ensure robustness.

3.1.4 Scraping Challenges

The dynamic and inconsistent structure of the IBM SkillsBuild platform introduces several challenges for automated extraction. While most metadata are statically rendered, elements like tags often load asynchronously, and sometimes may not appear at all. To address this, the scraper includes wait mechanisms with a timeout (e.g., 2 seconds); if tags do not load in time or are absent, parsing proceeds without them to ensure scraping continuity.

Navigating inside course modules to retrieve instructional content was infeasible due to non-repetitive layouts and dynamic DOM rendering. Consequently, the scraper is limited to extracting metadata available from course overview pages.

Language filtering is applied during link generation using the `&languages=en` URL parameter. This early-stage filter ensures alignment with the monolingual embedding pipeline and reduces noise from irrelevant content, eliminating the need for post-hoc language detection.

Web crawling also introduces legal and ethical considerations. Although IBM’s robots.txt disallows automated scraping, the data collection was conducted with verbal and project-level approval for academic, non-commercial use under supervision from both university and IBM project leads. No protected or private data was accessed.

To ensure responsible scraping, the project adhered to best practices from the literature (Olston & Najork, 2010; Khder, 2021), including:

- **Rate limiting** to prevent server overload
- **Restricting access** to publicly visible metadata only
- **Avoiding bypass** of authentication or protected areas

These safeguards ensured ethical compliance while enabling robust metadata extraction.

Challenge	Description	Mitigation Strategy
Dynamic tag loading	Tags load asynchronously, may not appear at all	Use wait mechanisms with 2s timeout
Inconsistent DOM structure	Courses are non-repetitive and dynamic	Scrape only overview metadata
Ethical and Legal Compliance	Crawling is disallowed in robots.txt	Data collection aligned with project scope approved by IBM
Multilingual content	Non-English courses may be scraped	Filter by URL parameter &languages=en

Table 1. Summary of scraping challenges and mitigation strategies used for IBM SkillsBuild.

This crawling pipeline ensures reliable data extraction while maintaining compliance with ethical and legal guidelines. Further details on potential risks and how they were addressed are documented in *Appendix A*.

3.2 Data Preprocessing and Cleaning

High-quality preprocessing is essential for ensuring the consistency, integrity, and utility of course metadata prior to embedding and database storage. The raw course metadata scraped from IBM SkillsBuild is initially stored in .json format, allowing structured access to fields during downstream processing. The subsequent cleaning phase ensures that both structured and unstructured fields are normalised, facilitating robust semantic indexing and query processing.

3.2.1 Cleaning and Extracting Structured fields

Structured fields, such as title, duration, and tags, are cleaned at the time of scraping using HTML tag stripping and DOM element filtering. Elements designed for screen readers or visual-only rendering (e.g., .sr-only) are removed to eliminate non-semantic noise. HTML parsing is handled using BeautifulSoup4, which provides efficient and reliable access to nested content structures (Crummy, 2020).

Descriptions, due to their higher complexity and formatting irregularities (e.g., embedded HTML, boilerplate text), are retained in raw form during scraping and cleaned later. All fields are cast to string type to ensure uniformity, especially in

cases where duration values may be presented as raw numbers ("90") or mixed strings ("1h 30min"). This approach enables consistent downstream parsing and simplifies batch processing.

3.2.2 Handling Incomplete or Corrupt Data

Data integrity is enforced by discarding entries that lack a valid URL, since the final URL segment serves as the unique course identifier (`course_id`). Missing core fields such as title or description are substituted with empty strings to prevent vector models from embedding placeholder tokens, which could introduce semantic noise or distort similarity calculations.

To preserve retrieval accuracy, incomplete numeric fields such as duration, ratings, or learner counts are assigned `None` values when absent, rather than defaulting to zero, which could falsely indicate neutral or baseline data. This conservative handling approach aligns with best practices in data cleaning for machine learning, where the avoidance of spurious data is critical for maintaining model robustness (Mitchell, 2018).

Moreover, as the scraping logic enforces English-language filtering directly in the search URLs (`&languages=en`), the need for downstream language detection is significantly reduced. This not only improves preprocessing speed but also guarantees consistency with the monolingual embedding model employed (all-MiniLM-L6-v2), which performs best on English-language inputs (Reimers and Gurevych, 2019).

3.2.3 Normalisation and standardisation

To ensure uniformity, all textual fields, such as titles, descriptions, and tags, are lowercased and cleaned using regular expression (regex)-based patterns. This removes boilerplate expressions like promotional or instructional text (e.g., "click to enrol", "earn a badge"), which are common in course metadata and irrelevant for semantic matching.

Durations are converted into a single unit (minutes) using regex token extraction, while ratings and learner counts are normalised using numeric parsing to maintain numerical consistency across records. For example, string-based ratings like "4.5 stars" are parsed into floating-point values on a 0–5 scale, aligned with industry-standard recommendation systems.

Course languages are stored using ISO 639-1 two-letter codes (e.g., "en" for English), supporting extensibility for multilingual support. Finally, each course's identifier (`course_id`) is derived from its URL slug, which guarantees global uniqueness and enables efficient primary-key indexing within the relational database schema.

Field	Raw Format Example	Processed Format	Method
Duration	"1h 30mins"	90	Regex -> Integer (minutes)

Star rating	“Average rating of 4.5 stars by 39 learners in the past 12 months.”	4.5	Regex -> float rating (0-5 scale)
number of learners	“1004 learners have completed this activity in the pas 12 months.”	1004	Regex -> Integer
Tags, Title,	“Cybersecurity Fundamentals (Earn a credential!)”	“cybersecurity fundamentals”	Regex, lowercase
Description	HTML with boilerplate	clean text, [“language”]	Regex, tag stripping, lowercase, Unicode normalisation (ftfy library)

Table 2. Summary of cleaning and normalisation methods used.

3.3 Hybrid Database Design

The system adopts a hybrid database architecture using PostgreSQL extended with the pgvector extension, enabling the integration of structured metadata with dense semantic embeddings. This design supports both traditional filtering and semantic similarity search within a unified query layer, an approach supported in hybrid information systems (Liu & Gawlick, 2015; Ma, Molnár & Tarcsi, 2022).

3.3.1 Structured Data: Schema-Based Storage

Structured metadata is stored in a well-defined relational schema within PostgreSQL. Each course is uniquely identified by `course_id`, and standard fields such as title, duration, and ratings are indexed to support efficient querying and filtering.

Field Name	Data Type	Description
<code>course_id</code>	TEXT (PK)	Unique identifier for each course (derived from URL segment).
<code>course_url</code>	TEXT	Full URL of the course on IBM SkillsBuild.
<code>category</code>	TEXT	Course category (e.g., cybersecurity, AI).
<code>type</code>	TEXT	Course type or format (e.g., video, module, badge).
<code>title</code>	TEXT	Cleaned course title.

duration	INTEGER	Duration of the course in minutes.
learners_amount	INTEGER	Number of learners enrolled in the course.
star_rating	REAL	Average course rating (0–5 scale).
star_num_rating	INTEGER	Number of users who rated the course.
description	TEXT	Cleaned course description.
tags	TEXT []	List of cleaned tags associated with the course.
languages	TEXT []	Languages in which the course is offered (ISO format).
embedding_input	TEXT	Concatenated string of title, description, and tags (used for embedding).
embedding_vector	VECTOR(384)	Dense vector embedding of course content for semantic search.

Table 3. Fields, data types and description of the PostgreSQL courses table.

3.3.2 Unstructured Data: Vector Embedding Storage

Unstructured content, course descriptions, titles, and tags, is encoded into 384-dimensional embeddings using the all-MiniLM-L6-v2 model (Reimers & Gurevych, 2019). These embeddings are stored using the pgvector extension, enabling efficient similarity search via cosine distance. Indexing on the vector column supports fast approximate nearest-neighbour queries within PostgreSQL.

3.3.3 Integration Strategy

All data is stored in a single unified table, allowing combined SQL queries that apply both structured filters (e.g., course duration, rating) and vector-based similarity conditions. This architecture simplifies query execution, avoids data duplication, and improves performance by minimising cross-system communication, key to supporting flexible and low-latency semantic search in educational applications.

3.4 Embedding Strategy

3.4.1 Model Selection

The embedding model used is all-MiniLM-L6-v2, part of the SentenceTransformers library (Reimers & Gurevych, 2019). This model produces 384-dimensional dense vectors, offering a strong balance between semantic accuracy and computational efficiency, key for real-time retrieval systems. It has

demonstrated robust performance on tasks from the BEIR benchmark (Thakur et al., 2021), making it well-suited for educational content retrieval.

3.4.2 Input Construction

Each course embedding is generated from a combined input comprising the course title, description, and tags. This approach captures both contextual and topical signals, enhancing semantic richness. The SentenceTransformers pipeline handles tokenisation and truncation automatically, preserving up to 256 tokens as per the model's limit.

Title	Description	Tags	Combined Text
Cybersecurity Fundamentals	Learn the basics of digital security	["Cybersecurity", "Fundamentals"]	title: cybersecurity fundamentals. description: learn the basics of digital security. tags: cybersecurity, fundamentals.

Table 4. Example of combined input to be used for vector embedding.

3.4.3 Embedding Generation

Embeddings are generated in batch using the model's `encode()` method and appended to the data pipeline. Both the embedding input string and its resulting vector are stored in PostgreSQL to support reproducibility, future fine-tuning, and optional baseline retrieval.

3.5 Semantic Search Design

3.5.1 Embedding-Based Retrieval

The search engine retrieves relevant courses by computing cosine similarity between embedded user queries and stored course embeddings using the `<=>` operator in pgvector (PostgreSQL). A configurable distance threshold is used to filter out semantically irrelevant results, and top-*k* courses are returned based on proximity. This allows for conceptual matching even when there is no lexical overlap, a limitation of traditional keyword-based systems (Robertson & Zaragoza, 2009).

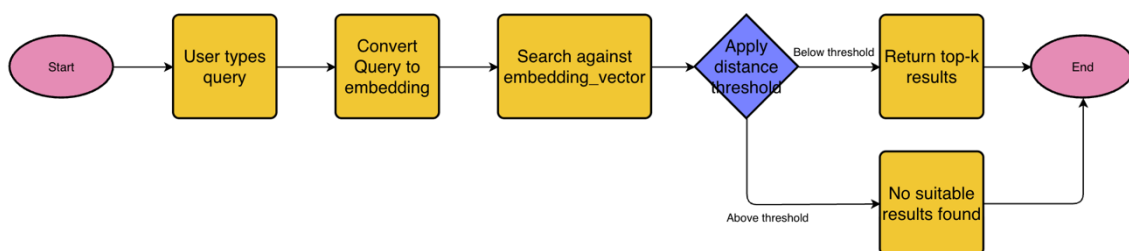


Figure 3. Figure Semantic search query processing workflow.

This diagram illustrates the end-to-end flow of the semantic retrieval system. A user query is embedded into a dense vector and compared against precomputed course embeddings. A distance threshold is applied to filter out semantically irrelevant results. If matches don't exceed the threshold, the top-k most similar courses are returned; otherwise, the system indicates that no suitable results were found.

3.5.2 Structured Filtering for Enhanced Retrieval

To improve result relevance and alignment with user constraints, structured filters can be applied based on course metadata such as language, rating, learner count, and duration. These filters are defined via SQL WHERE clauses, integrated into the same query as vector similarity logic, allowing unified execution across structured and unstructured dimensions. For example:

```
WHERE star_rating > 4.5 AND embedding_vector <=> :query_vector < :threshold
```

Code Snippet 1. Example WHERE clause combining structured filtering (by rating) and semantic similarity (via cosine distance).

Such filtering enables expressive, context-sensitive retrieval that scales well with growing datasets.

3.5.3 BM25 and Keyword Search (for evaluation only)

Although the deployed system uses only semantic search, two lexical baselines were implemented for evaluation: PostgreSQL's full-text search (via `to_tsvector`, `ts_rank_cd`) and a custom BM25+ scoring system in Python. These methods were used exclusively during benchmarking (see Chapter 5) to compare the semantic engine against traditional retrieval techniques.

Chapter 4: Implementation

This chapter describes the full technical implementation of the semantic search system developed for IBM SkillsBuild. It translates the design choices presented in the methodology into concrete software components, detailing the tools, modules, and workflows used across the data pipeline. The implementation is divided into six key areas: (1) web scraping and HTML parsing of course data; (2) metadata cleaning and normalisation; (3) semantic embedding generation; (4) hybrid database storage; (5) query execution across semantic and baseline retrieval models; and (6) deployment of the system via a FastAPI interface. Code references and utilities used in these steps are documented in Appendix C.

4.1 Web Scraping and HTML Parsing

The web scraping functionality is encapsulated in the `IBM_scraper.py` module, which leverages Selenium WebDriver to navigate dynamic web content and automate login procedures. Authentication is handled using credentials stored in a `.env` file, with the login automation logic detailed in *Appendix C.1*. Course discovery is initiated through dynamically constructed search URLs for each course category, based on parameters defined in the `config.json` file and assembled using custom URL-building functions, these are described in *Appendix C.2*.

Courses are extracted from search results using a multi-tab scraping strategy, implemented via the custom `__open_new_tab()` context manager, which manages browser tab switching through Selenium (*Appendix C.3*). The scraper iteratively loads all visible course cards, collects their URLs, and opens individual course pages for content extraction.

Each course page is parsed using the `parse_course_page()` function from `html_parser.py`, which uses BeautifulSoup to extract structured metadata such as course title, description, duration, tags, and user ratings (*Appendix C.4*). The helper function `extract_elements()` supports flexible content extraction, enabling attribute selection, raw HTML access, and nested tag removal.

Finally, the structured metadata is exported to both JSON and CSV formats using the `save_data()` function from `io_utils.py`, which wraps standard pandas export operations.

4.2 Data Cleaning and Normalisation

Raw course metadata is cleaned and normalised using a modular processing pipeline defined in `process_courses.py`. The cleaning process prepares the data for semantic embedding by improving text consistency, removing noise, and standardising metadata fields.

Key operations include:

- **Boilerplate and HTML cleaning:** `DescriptionCleaner` from `html_cleaner.py` decodes HTML, removes recurring instructional phrases (e.g., “Click here to enrol”), and extracts embedded language codes using `BeautifulSoup` and regular expressions (*Appendix C.5*).
- **Metadata extraction:** Functions in `metadata_extractor.py` extract numeric values such as learner counts, ratings, and review numbers from semi-structured text (*Appendix C.6*).
- **Title and tag normalisation:** `clean_title()` and `clean_tags()` from `text_cleaner.py` standardise case, punctuation, and formatting for downstream processing.
- **Duration conversion:** `convert_duration()` transforms natural language strings (e.g., “2 hours 30 minutes”) into total minutes, ensuring numerical consistency in filters and ranking.

A curated list of boilerplate phrases, loaded from `boilerplate_phrases.yml`, is compiled into a regex pattern to systematically strip platform-specific instructional content. This improves the semantic quality of course descriptions and reduces embedding noise.

All cleaned records are exported in JSON and CSV formats and stored in a processed data directory for subsequent embedding generation and insertion into the database for use in the semantic search engine.

4.3 Embedding Generation

Dense vector embeddings are generated using the `embed_courses.py` pipeline, which encodes each course’s metadata into a 384-dimensional semantic representation. The model is loaded via the `load_embedding_model()` function in `model_loader.py`, which retrieves the appropriate Sentence-BERT model (all-MiniLM-L6-v2) based on a `.env` configuration key, allowing the embedding model to be easily updated or replaced without modifying the pipeline code.

Each course record is transformed into a semantically meaningful input string using `build_combined_text()` from `text_utils.py` (*Appendix C.7*). This function concatenates the course title, description, and tags into a unified format optimised for embedding.

The core embedding logic is handled by `compute_embeddings()` in `embed_courses.py` (*Appendix C.8*). It applies the model to the combined input and appends the resulting vectors to the `DataFrame` under the `embedding_vector` column. The final dataset contains both the input text and its corresponding vector, which are then passed to the database layer for storage and retrieval.

4.4 Database Storage

Structured and vectorised course metadata is stored in a PostgreSQL database using a dedicated ingestion pipeline defined in `load_data.py`. The system uses SQLAlchemy to connect to the database, initialise the schema, and perform secure bulk inserts.

The database schema is defined in `sql_queries.py` and includes both standard course metadata (title, description, ratings, tags, etc.) and a 384-dimensional `embedding_vector` column of type `VECTOR(384)` enabled by the `pgvector` extension. This column supports fast similarity search based on cosine distance.

The table is created using the `CREATE_TABLE_QUERY`, shown in *Appendix C.9*, and ensures the required extension is installed. When data is inserted via the `INSERT_QUERY`, `ON CONFLICT` logic is used to update existing records based on the primary key `course_id`, thereby ensuring idempotent storage and avoiding duplication (see *Appendix C.10*).

Before insertion, the data undergoes deduplication and type standardisation through the `preprocess_dataframe()` function. Embeddings are computed using Sentence-BERT, and the final records are prepared with `prepare_row()` before being batch-inserted into the database.

All database parameters, including the URL and model name, are dynamically loaded from environment variables defined in the `.env` file, as configured via `config.py`.

4.5 Query Execution and Retrieval

The retrieval logic is defined primarily in `retrieve_courses.py` and supports three search modes: semantic search, TF-IDF-based keyword search, and custom BM25-style retrieval. All retrieval functions are designed to be modular and accept optional filtering parameters such as duration, level, and category.

4.5.1 Semantic Search

In semantic search mode, user queries are embedded using the same Sentence-BERT model (all-MiniLM-L6-v2) used during course indexing. The resulting 384-dimensional vector is computed in memory and then passed to PostgreSQL as a parameterised input. This vector is compared against stored course embeddings using the cosine distance operator `<=>` provided by the `pgvector` extension.

The semantic search logic is implemented in `retrieve_courses.py`, where the query vector is generated and optional structured filters (e.g., rating, duration, category) are dynamically incorporated based on user input. *Appendix C.11* shows how filters and thresholds are constructed, while *Appendix C.12* presents the SQL query template that performs the semantic similarity search using a `WITH` clause. This clause filters out results beyond the distance threshold and ranks the remaining entries by proximity to the query vector.

The system returns the top-*k* semantically relevant courses, effectively combining dense vector similarity with structured metadata constraints. If no course satisfies both the similarity and filtering criteria, an empty result is returned, allowing downstream components to handle fallback mechanisms as needed.

4.5.2 TF-IDF Keyword Search

TF-IDF-style keyword search is implemented using PostgreSQL's `to_tsvector` and `ts_rank_cd` functions across title, description, and tags. Documents are

weighted using field-level priorities (A for title, B for description, C for tags). Queries are converted to `plainto_tsquery` format for broader lexical matching.

The query is defined in `sql_queries.py` as `TF_IDF_SEARCH_QUERY` and executed from `retrieve_courses.py` through the `keyword_search()` function. The full SQL query is listed in *Appendix D.1* as part of the evaluation framework.

4.5.3 BM25-style Search

The custom BM25 implementation, located in `bm25.py`, performs semantic preprocessing using `spaCy` and `WordNet`. It lemmatises input terms, expands queries with `WordNet` synonyms, filters matching documents via SQL `ILIKE` conditions, and applies a BM25 scoring function across document embeddings.

Unlike the database-driven methods, the BM25 pipeline loads candidate rows into memory, preprocesses them with NLP, and expands queries using `WordNet` synonyms. Relevance scores are computed using a custom BM25 scoring function. The full retrieval and scoring workflow is shown in *Appendix D.4*, with text preprocessing and synonym expansion detailed in *Appendix D.2*, and the BM25 scoring function in *Appendix D.3* as part of the evaluation framework.

4.5.4 Execution and Output

All three query modes are executed using `SQLAlchemy`, and the results are returned as lists of Python dictionaries containing course metadata and, where relevant, a distance, rank, or `bm25_score` value. These functions are used during both interactive testing and offline evaluation to benchmark relevance, precision, and ranking performance.

4.6 Configuration and Utilities

The project is supported by a set of modular utility files that abstract configuration loading, browser automation, data I/O, and text preprocessing:

- **config.py** loads project settings from a JSON file and environmental variables and provides helper functions for database engine creation and boilerplate phrase loading.
- **selenium_utils.py** defines reusable browser control functions for dynamic content loading, login automation, and interaction with interface elements such as "Show More" buttons.
- **io_utils.py** handles batch saving and loading of scraped course data in both JSON and CSV formats.
- **text_utils.py** includes text preprocessing logic such as `build_combined_text()` used for semantic embedding input (*Appendix C.7*).
- **model_loader.py** loads the Sentence-BERT embedding model used for both semantic indexing and query retrieval. The specific model version is selected dynamically based on a configuration key (`EMBEDDING_MODEL_NAME`) defined in the `.env` file.

This modular architecture improves code clarity and maintainability by decoupling core functionalities into dedicated components. It allows for easy adaptation and extension, such as swapping embedding models, adjusting scraping parameters,

or reconfiguring database access, without modifying the main pipeline logic. This structure also facilitates downstream integration with additional modules like retrieval-augmented generation (RAG) or user-facing interfaces, ensuring long-term scalability and flexibility.

4.7 API Deployment

To enable real-time access to the semantic search engine and facilitate integration with external systems, the solution is exposed via a RESTful API developed using FastAPI, a lightweight, high-performance web framework selected for its native support for asynchronous operations, Pydantic-based type validation, and automatic OpenAPI documentation generation (Tiangolo, 2022; OpenAPI Initiative, 2023).

The API defines two primary endpoints:

- **/api/v1/courses/search/semantic:** Accepts a natural language query along with optional filters (e.g. course duration, rating, category), and returns a ranked list of semantically similar courses based on vector similarity using pgvector.
- **/api/v1/chat:** Implements a Retrieval-Augmented Generation (RAG) pipeline by combining semantic retrieval with a Large Language Model (LLM). This endpoint accepts a sequence of prior chat messages (i.e. chat history) and uses the most recent user query to retrieve top-matching courses, which are then passed, along with the full conversational context, to the LLM for natural language generation.

Structured request and response models, including input validation and filtering logic, are defined using SQLAlchemy, a SQLAlchemy-compatible ORM that integrates cleanly with FastAPI and provides type-safe, declarative schema definitions.

The API supports:

- **Range-based filtering** for numeric fields such as duration, star_rating, and learners_amount.
- **Categorical filtering** to narrow search results by domain-specific course topics (e.g. 'cloud_computing', 'cybersecurity').

A lifespan context manager is used to initialise and load all core system components at application startup, including the embedding model, language model, tokenizer, and database engine. Configuration is fully environment-driven via .env keys to support portability and secure deployment.

The implementation of the semantic search endpoint and its underlying filter and model handling logic is provided in *Appendix C.13*, while the supporting request and response models used for filtering and output formatting are detailed in *Appendix C.14*.

Chapter 5: Evaluation

This chapter presents the evaluation strategy used to assess the performance of the *SkillFindr* semantic search engine. The primary objective is to determine the system's ability to retrieve relevant educational content in response to natural language queries and to benchmark its performance against two traditional baselines: keyword-based search and the BM25 retrieval model. A combination of quantitative (metric-based) and qualitative (example-based) evaluation is conducted. The evaluation also supports answering the project's core research questions concerning semantic retrieval, metadata integration, and the system's impact on course discoverability.

5.1 Testing Methods

This chapter evaluates the semantic retrieval engine forming the core of the SkillFindr system, which will ultimately support the full learning assistant architecture. It details the systems under comparison, the construction of a domain-specific evaluation dataset, the retrieval metrics used, and limitations inherent to the evaluation process.

5.1.1 Evaluation Objectives

The goal of this evaluation is to assess the relevance, ranking quality, and semantic accuracy of course recommendations returned by the semantic search engine in response to natural language queries. The evaluation:

- Benchmarks the system against two traditional baselines: keyword-based matching and BM25.
- Assesses the performance of a hybrid architecture that integrates both structured (e.g., duration, tags) and unstructured (e.g., course descriptions) data.
- Supports answering whether semantic search improves course discoverability on the IBM SkillsBuild platform.

5.1.2 Systems Compared

Three search systems were implemented and evaluated:

- **Semantic Search Engine:** A dense vector-based retrieval system using *all-MiniLM-L6-v2*, a pre-trained Sentence-BERT model (Reimers & Gurevych, 2019), for generating 384-dimensional embeddings of both user queries and course content. Embeddings are stored in PostgreSQL using the pgvector extension, and results are ranked using cosine similarity.
- **Keyword-Based Search (Full-text Matching):** Implemented using PostgreSQL's full-text search tools: `to_tsvector`, `plainto_tsquery`, and ranked using `ts_rank_cd`. This system indexes course metadata (title, description, tags) and retrieves matches based on token overlap.
- **BM25 Search Engine:** A probabilistic ranking system implemented in Python using the BM25+ algorithm, which accounts for term frequency,

document length normalisation, and inverse document frequency (Robertson & Zaragoza, 2009). This approach is widely used in classical information retrieval systems.

The prediction functions used to collect results per system are described in *Appendix C*.

BM25 and keyword search often returned few or no results for natural language queries, an expected limitation that is reflected in the Coverage metric and discussed further in Section 5.3. This occurs because keyword-based retrieval methods depend on exact or partial lexical matches between query terms and document content. Natural language queries often contain implicit intent, varied phrasing, or abstract concepts that fail to align lexically with course metadata, resulting in low or null matches. Semantic models overcome this by embedding meaning rather than surface form.

5.1.3 Evaluation Dataset

A custom evaluation dataset was manually created to serve as ground truth. This was necessary due to the lack of existing labelled benchmarks for the IBM SkillsBuild platform and the project's reliance on scraped data.

- **Query Design:** A total of 50 natural language queries were written by hand to reflect realistic user intents. Queries span multiple educational domains (e.g., *Cybersecurity*, *Machine Learning*, *Agile Methodologies*, *Web Development*, *Blockchain*) and varying user goals (e.g., beginner orientation, career guidance, certification, skill-building). Keyword-only or overly specific queries were avoided to better simulate real use cases. A subset of representative queries is provided in *Appendix E* to illustrate the diversity of topics and intent types evaluated.
- **Relevance Judgement:** For each query, 20–30 potentially relevant courses were retrieved using all three systems with high recall thresholds. Each retrieved course was then manually annotated using a 3-point relevance scale:
 - 2 – Highly relevant
 - 1 – Somewhat relevant
 - 0 – Not relevant

These relevance scores reflect the semantic match between the course content and query intent, considering level, topic, and objectives.

- **Data Format:** The dataset is stored in JSON format. Each entry maps a query to a list of `course_ids` and their relevance scores. An example is shown below:

```
"career pathways in blockchain development": [
  {
    "course_id": "URL-QZUQ_TYBHUG",
    "relevance": 2
  },
  {
    "course_id": "CNL_LCB_1567443539771",
    "relevance": 1
  },
]
```

```
{
  "course_id": "ILB-NKQGKQQPRDJD4D7Q",
  "relevance": 0
}
```

Code Snippet 2. Test dataset entry example (JSON)

This dataset serves as the ground truth for computing Precision, Recall, MRR, and nDCG, ensuring consistency and reproducibility across experiments.

5.1.4 Metrics Used

To evaluate the retrieval quality of each search method, the following standard information retrieval metrics were computed:

- **Precision@k**: The fraction of relevant items among the top-k retrieved. High precision indicates that top results are relevant to the query.
- **Recall@k**: The fraction of all relevant documents that appear in the top-k retrieved results. Measures coverage of relevant content.
- **Mean Reciprocal Rank (MRR)**: The average of the reciprocal ranks of the first relevant document across all queries. Indicates how early in the ranked list the first relevant result appears.
- **Mean Average Precision (MAP@k)**: The mean of the average precision scores for each query, computed over the top-k retrieved items. It balances precision across different ranks and queries, providing a single metric that rewards both relevance and rank.
- **Normalized Discounted Cumulative Gain (nDCG@k)**: A graded relevance metric that considers the position and relevance score of each retrieved item. Rewards placing more relevant results higher in the list.
- **Coverage Rate**: The percentage of queries for which the system returned at least one result. This is crucial in scenarios where traditional methods (BM25, keyword) may fail to return any result for natural language queries.
- **Response Time**: While not the focus of this evaluation, response time was recorded for each system to compare basic computational performance.

All metrics were implemented in Python; see *Appendix D.5* for full metric functions.

5.1.5 Limitations of the Method

Despite efforts to ensure objectivity, the evaluation process is subject to several limitations:

- **Subjectivity in Judgements**: Manual relevance annotation is inherently subjective. To reduce bias, each query was annotated after reviewing a wide range of retrieved results from all systems and cross-comparing content.
- **High Recall Thresholds**: To minimise false negatives (i.e., missed relevant courses), a high retrieval limit (20–30 per query) was used before filtering and ranking. This ensures that borderline-relevant courses were not overlooked.
- **Absence of Real User Feedback**: The evaluation relies on proxy metrics and manual annotations. No live user testing or A/B experiments were

conducted. While proxy metrics are standard in IR research, they may not fully capture user satisfaction or usability.

- **Static Query Set:** All queries were predefined and do not reflect evolving or adaptive user behaviour. A dynamic query set could further validate the generalisability of results.

5.2 Results

This section presents the quantitative and qualitative outcomes of the system evaluation conducted over 50 manually annotated queries at a cutoff of $K = 20$. Results are compared across the three retrieval methods: Semantic Search, Keyword Search, and BM25.

5.2.1 Quantitative Metrics

Table 5 summarises the core performance metrics for each system at $K=20$. The semantic search engine clearly outperforms both traditional baselines across all metrics.

Metric	Semantic	Keyword	BM25
Precision@20	0.295	0.002	0.003
Recall@20	0.899	0.012	0.008
MRR	0.843	0.04	0.04
MAP@20	0.712	0.012	0.008
NDCG@20	0.781	0.015	0.013
Coverage	100%	4.00%	6.00%
Avg Latency	27.6 ms	62.3 ms	48.4 ms

Table 5. Information retrieval metrics at $k=20$

Each metric underscores a specific strength of the semantic search engine:

- **Precision@20** (0.295) shows that nearly 30% of top results are highly relevant, semantic ranking surfaces quality early.
- **Recall@20** (0.899) indicates broad coverage, retrieving almost all relevant courses, even with minimal lexical overlap.
- **MRR** (0.843) reflects that the first relevant result often appears near the top, key for user satisfaction.

- **MAP@20** (0.712) shows consistent ranking of relevant results, not just occasional hits.
- **nDCG@20** (0.781) confirms effective ranking, rewarding relevance at higher positions.

In contrast, BM25 and keyword search perform near zero across all metrics, unable to capture user intent in natural language queries, leading to poor Precision, Recall, and low result return rates.

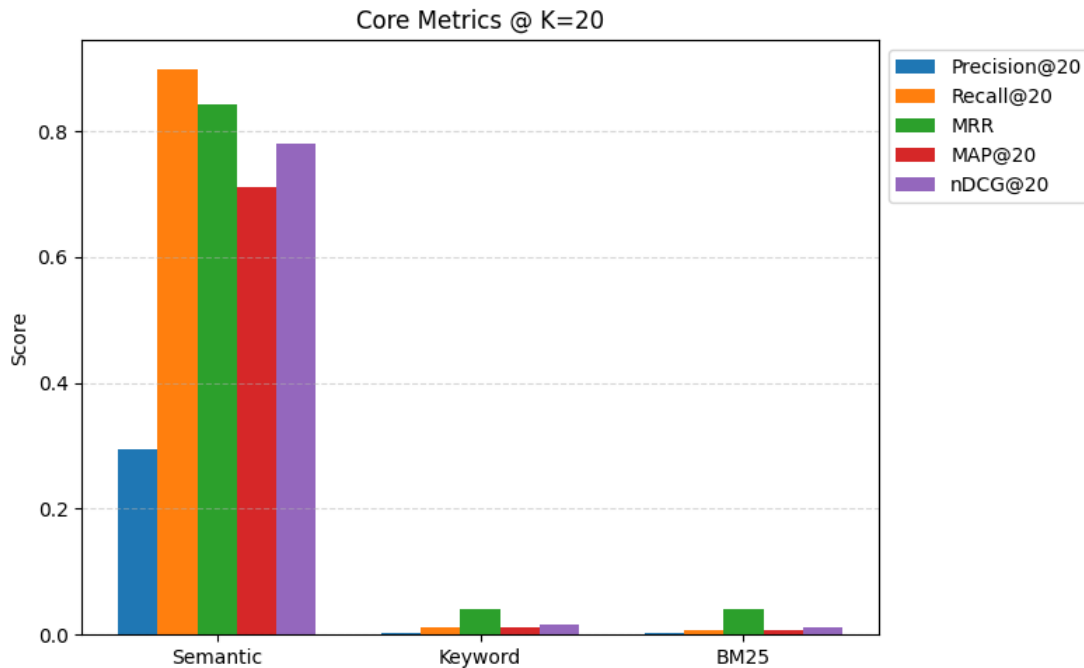


Figure 4. Presents a comparison of the main retrieval metrics, *Precision@20*, *Recall@20*, *MRR*, *MAP@20*, and *nDCG@20*, across semantic search, BM25, and keyword search.

The semantic system consistently outperforms both baselines, achieving significantly higher scores across all dimensions. Its high *Recall@20* (0.899) and *MRR* (0.843) demonstrate both breadth of retrieval and early relevance, while *MAP@20* and *nDCG@20* confirm effective ranking. In contrast, BM25 and keyword search return few relevant results, validating the semantic model's superiority in capturing user intent from natural language queries.

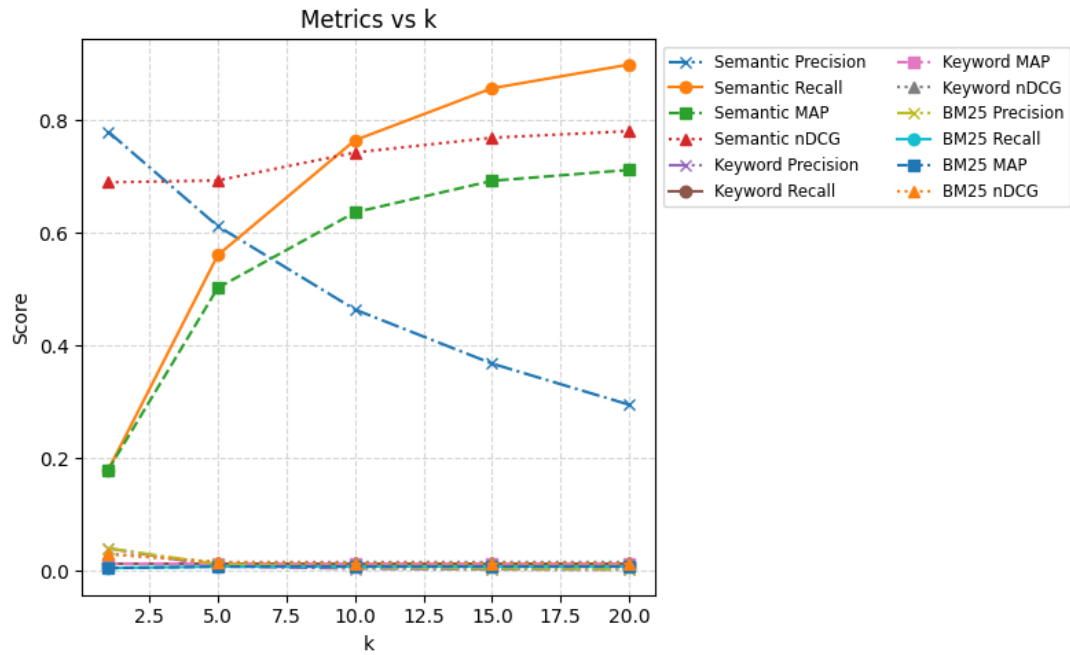


Figure 5. Shows how Precision, Recall, MAP, and nDCG vary as the retrieval cutoff k increases, comparing semantic search to the baseline systems.

The metrics for semantic search steadily increase with larger k -values, suggesting robust scalability. In contrast, BM25 and keyword models remain flat, indicating that additional results do not improve relevance, likely due to their limited understanding of semantic context

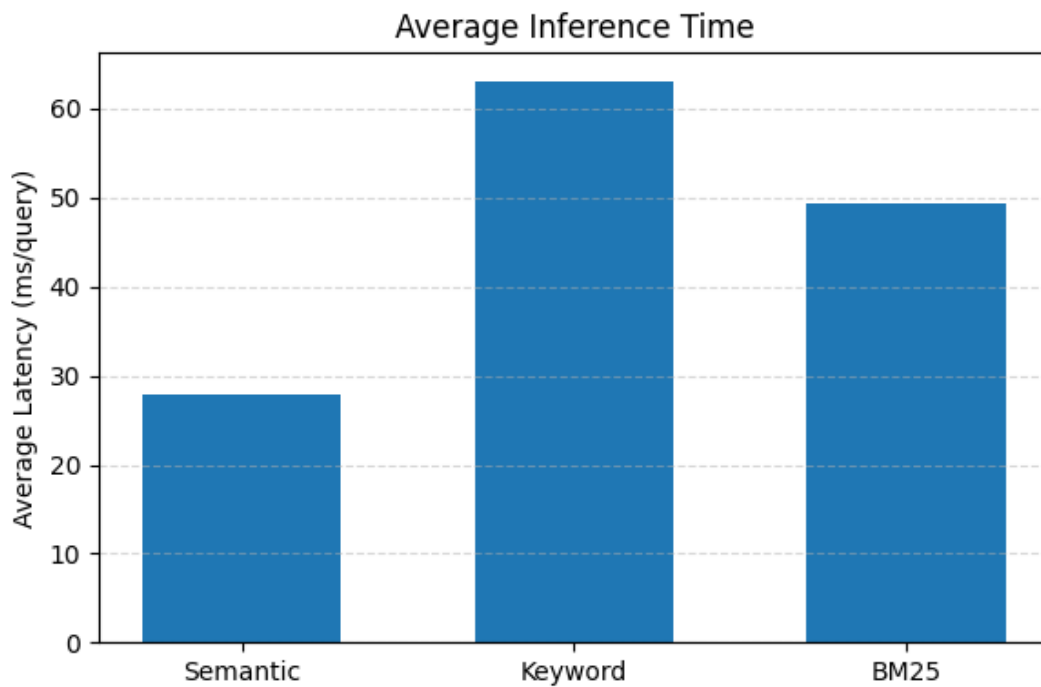


Figure 6. Depicts the average response time (in milliseconds) per query for each search method.

Semantic search has the lowest average latency at 27.6 ms, significantly outperforming both keyword (62.3 ms) and BM25 (48.4 ms). This demonstrates that semantic retrieval is not only more accurate but also more efficient, making it viable for real-time educational search applications.

Evaluation routines, visualisation functions, and performance logging utilities are available in *Appendix D.6* through *D.8*.

Together, these results demonstrate that semantic search provides both higher quality and more consistent retrieval in response to natural language queries.

5.2.2 Qualitative Example

While quantitative metrics offer a high-level view of performance, they don't capture how well each system meets real user needs. This section compares four representative queries across the three retrieval systems to illustrate practical differences. Each example includes the top three results and their relevance scores, covering a range of intents from skill development to certification and career guidance.

Query 1: Complete overview of cloud computing for beginners

This query targets entry-level learners looking for a structured and beginner-friendly introduction to cloud computing concepts.

System	Rank	Course Title	Relevance
Semantic Search	1	introduction to cloud	2
	2	introduction to cloud computing	2
	3	cloud computing characteristics, history, and trends	1
BM25 Search	1-3	No results	X
Keyword Search	1-3	No results	X

Table 6. Search results for query 1: beginner artificial intelligence course for non-programmers.

Query 2: Get certified in basic cybersecurity principles

This query reflects a goal-oriented learner seeking an entry-level certification in cybersecurity, with an emphasis on foundational principles.

System	Rank	Course Title	Relevance
--------	------	--------------	-----------

Semantic Search	1	vetjobs & covered6 cyber basics	2
	2	ibm cybersecurity analyst professional certificate	2
	3	fortinet certified fundamentals (fcf)	2
BM25 Search	1-3	No results	X
Keyword Search	1-3	No results	X

Table 7. Search results for query 2: get certified in basic cybersecurity principles.

Query 3: Introduction to SQL queries for data analysis

This is a short, keyword-style query focused on learning basic SQL for analysing data, a common and well-scoped technical skill.

System	Rank	Course Title	Relevance
Semantic Search	1	introduction to databases and sql querying	2
	2	sql and relational databases 101	2
	3	steps in data processing ibm skillsbuild	0
BM25 Search	1	overview of data tools and languages	1
	2-3	No results	X
Keyword Search	1-3	No results	X

Table 8. Search results for query 3: introduction to SQL queries for data analysis.

Query 4: Create interactive dashboards with Tableau

This task-based query targets learners interested in using the Tableau software to build interactive data dashboards, combining tool-specific and practical objectives.

System	Rank	Course Title	Relevance
--------	------	--------------	-----------

Semantic Search	1	introduction to tableau desktop	2
	2	introducing tableau prep builder	1
	3	sharing and collaborating in tableau cloud	1
BM25 Search	1-3	No results	X
Keyword Search	1	introduction to tableau desktop	2
	2-3	No results	X

Table 9. Search results for query 4: create interactive dashboards with Tableau

Across all four examples, semantic search consistently returned highly relevant courses aligned with the user's intent, even when query wording did not match course titles verbatim. In contrast, BM25 and keyword search often failed to return results, particularly for conceptually phrased or longer queries. This reinforces the earlier quantitative findings that semantic search offers superior performance in both coverage and relevance for natural language educational queries.

5.3 Discussion and Research Question Analysis

This section interprets the results and evaluates how they address the core research questions posed in Chapter 1.

5.3.1 Research Question 1: How can semantic search enhance course discoverability on the IBM SkillsBuild platform?

The results strongly support the hypothesis that semantic search substantially improves course discoverability compared to traditional keyword-based systems. With a Recall@20 of 0.899, the semantic search engine retrieved most relevant courses across diverse query types. In contrast, BM25 and keyword search achieved less than 1.2% recall, often returning no results at all for natural language queries.

Moreover, the Mean Reciprocal Rank (MRR) of 0.843 and MAP@20 of 0.712 demonstrate that relevant courses were not only retrieved but also ranked effectively near the top. The nDCG@20 score of 0.781 further highlights the quality of ranking, with relevant courses given appropriate weight according to their position in the result list.

Coverage was another critical differentiator: semantic search achieved 100% coverage, returning at least one result for most queries, while BM25 and keyword search managed only 6% and 4% respectively. This is a key factor in improving user experience, as it ensures users are presented with meaningful responses to their queries.

These results confirm that semantic embeddings enable the system to understand user intent rather than rely solely on lexical matches. This capability is especially important in educational contexts, where users often pose conceptual or instructional queries that do not directly match course titles or metadata.

Conclusion: Semantic search significantly enhances course discoverability by increasing both the quantity and quality of results returned for natural language queries. It addresses limitations in keyword-based systems and aligns with the platform's goal of improving user experience.

5.3.2 Research Question 2: How can web crawling strategies be optimised for reliable and scalable metadata extraction in educational platforms like IBM SkillsBuild?

The web crawling strategy implemented in this project proved both reliable and scalable for the extraction of course metadata. Using Selenium and BeautifulSoup, the scraper was able to navigate the dynamically rendered SkillsBuild interface and extract structured fields such as course title, description, tags, and duration. Over 1200 courses were successfully scraped, demonstrating the strategy's ability to scale across categories.

To address inconsistencies in the DOM and dynamic loading issues, the crawler implemented:

- Wait mechanisms and timeouts for asynchronous content,
- A language filter to restrict results to English courses,
- Robust parsing and normalisation of fields such as durations, ratings, and tag sets.

Ethical considerations were addressed through project-level approval from IBM and the university, allowing academic, non-commercial scraping despite robots.txt restrictions. The crawler followed established best practices (Olston & Najork, 2010; Khder, 2021), including rate limiting, access to public metadata only, and strict avoidance of protected content, ensuring compliant and responsible data collection.

Conclusion: The project successfully developed a crawling pipeline that is ethically compliant, dynamically robust, and adaptable to content changes on educational platforms. This strategy supports scalable metadata collection, essential for maintaining up-to-date search functionality.

5.3.3 Research Question 3: How can hybrid database systems integrate structured and unstructured data for effective semantic search?

The hybrid database architecture implemented in the semantic search component of *SkillFindr* demonstrates a successful integration of structured and unstructured data through a combination of:

- **Schema-based relational storage** for fields like course title, rating, and duration,
- **Vector-based pgvector storage** for dense embeddings representing course descriptions and tags.

This approach enabled complex queries that combined:

- **Structured filtering** (e.g., “rating > 4.5”),
- **Semantic similarity ranking** (via cosine distance in the vector space).

Despite storing 384-dimensional vectors and metadata in the same table, query performance remained efficient, averaging 27.6 ms per semantic query, which is faster than both keyword (62.3 ms) and BM25 (48.4 ms). This validates the scalability and responsiveness of the hybrid model.

Moreover, the architecture allowed future extensibility, including the potential for Retrieval-Augmented Generation (RAG), which would combine dense retrieval with generative LLM-based responses.

Conclusion: The hybrid database design proved effective for combining structured metadata with unstructured course content. It facilitated accurate semantic matching while preserving the ability to apply filters and constraints, making it ideal for educational retrieval applications.

5.4 Summary

This chapter presented a comprehensive evaluation of the semantic search engine developed as part of the SkillFindr in comparison with traditional keyword-based and BM25 search approaches. Through a manually curated dataset of 50 natural language queries, the semantic system demonstrated significantly higher performance across all key retrieval metrics, including Recall@20 (0.899), MRR (0.843), and MAP@20 (0.712). The system also achieved a notably higher coverage rate (100%) and lower latency (27.6 ms/query), establishing it as both effective and efficient.

The evaluation methodology combined quantitative metrics with qualitative examples to illustrate the system’s ability to interpret user intent and return semantically relevant course results. In contrast, both BM25 and keyword search returned few or no results for most queries, highlighting the limitations of lexical retrieval on natural language educational queries.

Each of the project’s three research questions was directly addressed through this evaluation:

1. Semantic search significantly improves course discoverability.
2. The web crawler reliably extracted metadata at scale.
3. A hybrid database design proved successful in unifying structured and unstructured data for semantic retrieval.

The chapter establishes a strong empirical foundation for the project’s conclusions and future work, demonstrating that the semantic search engine within the SkillFindr system offers a practical and impactful solution to the discoverability challenges faced by platforms like IBM SkillsBuild.

Chapter 6: Conclusion

6.1 Overview

This project set out to address a key usability challenge faced by educational platforms like IBM SkillsBuild: enabling users to efficiently discover relevant learning content using natural language queries. To solve this, a semantic search engine was designed, developed, and evaluated as the core retrieval component of the broader *SkillFindr* system. It integrates a semantic search engine using sentence embeddings, a hybrid relational and vector database structure, and a scalable metadata extraction pipeline. The project aimed to test whether a semantic search approach could outperform traditional retrieval methods such as keyword search and BM25, and whether it could be deployed in a way that is practical and scalable for real-world use. The following sections reflect on the system's achievements, challenges, and potential for future development.

6.2 Summary of Achievements

This project achieved its primary objectives and demonstrated the viability of semantic retrieval for educational content discovery:

- **Semantic Search Engine Implementation:** A vector-based retrieval system was developed using the pre-trained all-MiniLM-L6-v2 model and integrated with PostgreSQL via the pgvector extension. This enabled efficient cosine similarity ranking between embedded course representations and natural language queries.
- **Hybrid Database Design:** A unified schema was implemented to support both structured metadata and dense embeddings, facilitating seamless integration of full-text and semantic search capabilities within a single database layer.
- **Metadata Acquisition Pipeline:** A Selenium-based scraping framework was used to extract over 1,200 course records from the IBM SkillsBuild platform, capturing both structured (e.g., duration, rating, tags) and unstructured (e.g., descriptions) data.
- **Evaluation Framework:** A manually annotated dataset comprising 50 natural language queries was constructed to evaluate retrieval quality. Semantic search consistently outperformed keyword and BM25 baselines across standard metrics (e.g., Recall@20 = 0.899, MRR = 0.843), and exhibited superior coverage and response time.
- **API Deployment and RAG Integration:** The system was deployed using FastAPI, exposing modular endpoints for semantic search and Retrieval-Augmented Generation (RAG). This architecture enables real-time integration with LLM-based dialogue systems for contextualised educational recommendations.

Risks related to data extraction, model performance, and compliance were anticipated and mitigated through early planning. A full risk assessment is presented in *Appendix A*.

These contributions collectively demonstrate that semantic search improves retrieval effectiveness, system responsiveness, and alignment with user intent in educational search platforms.

6.3 Reflection on Research Questions

Each of the project's three research questions was addressed through design, implementation, and evaluation.

RQ1: How can semantic search enhance course discoverability on the IBM SkillsBuild platform?

The evaluation results showed a dramatic improvement in both the quality and quantity of retrieved results. The semantic search engine achieved over 84% MRR and outperformed lexical baselines by a wide margin, indicating stronger alignment with user intent and improved discoverability.

RQ2: How can web crawling strategies be optimised for reliable and scalable metadata extraction?

The implemented web crawler was capable of navigating dynamic elements and extracting structured metadata from the SkillsBuild platform at scale. With IBM's permission, the scraper was run ethically and collected a rich dataset, forming the basis for effective indexing.

RQ3: How can hybrid database systems integrate structured and unstructured data for effective semantic search?

The integration of relational fields (e.g., course ratings, durations) with vector embeddings for unstructured descriptions enabled flexible and efficient querying. The system supported fast similarity ranking while maintaining filtering capabilities, demonstrating the practical success of the hybrid architecture.

6.4 Limitations

While the project achieved its main goals, several limitations were encountered:

- **Subjectivity in Relevance Judgements:** Manual labelling introduces bias, even with a fixed rubric. Future iterations could involve multiple raters or user-generated feedback.
- **Limited Query Set:** The evaluation was based on 50 static queries. This does not capture real-time user interactions or adaptive querying behaviour.
- **Lack of Fine-Tuning:** The embeddings used were pre-trained and not domain adapted. A fine-tuned model might yield even higher performance.
- **English-Only Indexing:** Only English-language content was indexed and tested, excluding multilingual courses available on the platform.
- **No Live User Feedback:** The system was evaluated offline, without testing user satisfaction or incorporating A/B testing in production.

6.5 Future Work

There are multiple directions in which this project can be extended:

1. **Domain-Specific Embedding Fine-Tuning:** Train or fine-tune embeddings on educational corpora to improve semantic alignment with learning content.
2. **Live User Testing and Feedback Integration:** Deploy the system in a controlled user environment to collect interaction data and satisfaction scores, enabling further refinement.
3. **Multilingual Support:** Extend the index to include French, Spanish, and other non-English content available on SkillsBuild by adopting multilingual embedding models such as XLM-RoBERTa (Conneau et al., 2020). This would enable cross-lingual semantic retrieval, allowing users to search and receive results in their preferred language, thus broadening the platform's accessibility and inclusivity.
4. **Course Recommendation Engine:** Incorporate user profiles and behavioural data to deliver personalised recommendations beyond static search.
5. **Adaptive Ranking Based on Click-through:** Train a lightweight ranking model that adjusts scores based on observed user preferences and feedback.

These enhancements would further empower platforms like IBM SkillsBuild to support learners with diverse goals and backgrounds, improving the accessibility and personalisation of educational content.

References

Website References

1. IBM SkillsBuild Course Catalog (n.d.). Available at: <https://skillsbuild.org/students/course-catalog> [Accessed 22 November 2024].
2. IBM SkillsBuild Digital Credentials (n.d.). Available at: <https://skillsbuild.org/students/digital-credentials> [Accessed 22 November 2024].
3. IBM SkillsBuild (n.d.). *About IBM SkillsBuild*. Available at: <https://skillsbuild.org/about> [Accessed 22 November 2024].
4. OpenAPI Initiative, 2023. *OpenAPI Specification*. [online] Available at: <https://www.openapis.org/> [Accessed 21 Apr. 2025].

Books and Foundational Texts

1. Manning, C.D., Raghavan, P. and Schütze, H., 2008. *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
2. Olston, C. and Najork, M., 2010. *Web Crawling. Foundations and Trends® in Information Retrieval*, 4(3), pp.175–246.
3. Robertson, S.E. and Zaragoza, H., 2009. *The Probabilistic Relevance Framework: BM25 and Beyond*. In: *Foundations and Trends® in Information Retrieval*, 3(4), pp.333–389. [online] Available at: <https://doi.org/10.1561/15000000019> [Accessed 27 November 2024].
4. Roelleke, T., 2013. *Information Retrieval Models: Foundations and Relationships*. 1st ed. San Rafael, California: Morgan & Claypool Publishers.
5. Mitchell, R., 2018. *Web Scraping with Python: Collecting Data from the Modern Web*. 2nd ed. Sebastopol, CA: O'Reilly Media.

Practical Tools and Documentation

1. OpenAI, 2023. *Embeddings API: Ada Model Documentation*. [online] Available at: <https://platform.openai.com/docs/guides/embeddings> [Accessed 20 November 2024].
2. Crummy, K., 2020. *Beautiful Soup Documentation*. [online] Available at: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> [Accessed 15 October 2024].
3. Selenium Documentation, 2024. *Selenium Documentation*. [online] Available at: <https://www.selenium.dev/documentation/en/> [Accessed 15 October 2024].
4. Tiangolo, S., 2022. *FastAPI Documentation*. [online] Available at: <https://fastapi.tiangolo.com/> [Accessed 18 Apr. 2025].
5. pgvector, 2023. *pgvector: Open-source vector similarity search for Postgres*. [online] GitHub. Available at: <https://github.com/pgvector/pgvector> [Accessed 21 April 2025].

Other References

1. Beltagy, I., Lo, K. and Cohan, A., 2019. *SciBERT: A Pretrained Language Model for Scientific Text*. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. [online] Available at: <https://arxiv.org/abs/1903.10676> [Accessed 26 November 2024].
2. Carbonell, J. and Goldstein, J., 1998. *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*. In: *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. [online] Available at: <https://dl.acm.org/doi/10.1145/290941.291025> [Accessed 27 November 2024].
3. Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2019. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. [online] Available at: <https://arxiv.org/abs/1810.04805> [Accessed 21 November 2024].
4. Khder, M.A., 2021. *Web Scraping or Web Crawling: State of Art, Techniques, Approaches and Application*. *International Journal of Advance Soft Computing and Applications*, 13(3), pp.145–164. DOI: <https://doi.org/10.15849/IJASCA.211128.11> [Accessed 27 November 2024].
5. Kraska, T., Beutel, A., Chi, E.H., Dean, J., and Polyzotis, N., 2018. The Case for Learned Index Structures. *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pp. 489–504. Available at: <https://doi.org/10.1145/3183713.3196909> [Accessed 27 November 2024].
6. Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C.H. and Kang, J., 2020. *BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining*. *Bioinformatics*, 36(4), pp.1234–1240. DOI: <https://doi.org/10.1093/bioinformatics/btz682>.
7. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D., 2020. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. [online] Available at: <https://arxiv.org/abs/2005.11401> [Accessed 25 November 2024].
8. Liu, Z.H. and Gawlick, D., 2015. *Management of Flexible Schema Data in RDBMSs—Opportunities and Limitations for NoSQL*.
9. Ma, C., Molnár, B. and Tarcsi, Á., 2022. *Knowledge-Enriched Schema Matching Framework for Heterogeneous Data Integration*.
10. Johnson, J., Douze, M. and Jégou, H., 2019. *Billion-scale similarity search with GPUs*. *IEEE Transactions on Big Data*, 7(3), pp.535–547. Available at: <https://doi.org/10.1109/TBDATA.2019.2921572> [Accessed 1 May 2025].
11. Maryamah, M., Irfani, M.M., Raharjo, E.B.T., et al., 2024. Chatbots in Academia: A Retrieval-Augmented Generation Approach for Improved Efficient Information Access. *2024 16th International Conference on Knowledge and Smart Technology (KST)*. IEEE. Available at:

- <https://doi.org/10.1109/KST61284.2024.10> [Accessed 20 November 2024].
12. Muennighoff, N., 2022. SGPT: GPT Sentence Embeddings for Semantic Search. *arXiv preprint arXiv:2202.08904*. Available at: <https://arxiv.org/abs/2202.08904> [Accessed 27 November 2024].
 13. Muennighoff, N., Tazi, N., Magne, L. and Reimers, N., 2022. *MTEB: Massive Text Embedding Benchmark*. [online] Available at: <https://arxiv.org/abs/2210.07316> [Accessed 26 November 2024].
 14. Nguyen, Z., Annunziata, A., Dinh, V.L.S., et al., 2024. Enhancing Q&A with Domain-Specific Fine-Tuning and Iterative Reasoning: A Comparative Study. *arXiv preprint arXiv:2404.11792*. Available at: <https://arxiv.org/abs/2404.11792> [Accessed 27 November 2024].
 15. Reimers, N. and Gurevych, I., 2019. *Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks*. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [online] Available at: <https://arxiv.org/abs/1908.10084> [Accessed 21 November 2024].
 16. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A. and Gurevych, I., 2021. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [online] Available at: <https://arxiv.org/abs/2104.08663> [Accessed 26 November 2024].
 17. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. *Attention is All You Need*. In: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30. [online] Available at: <https://arxiv.org/abs/1706.03762> [Accessed 20 November 2024].
 18. Yoo, H., Kang, M., and Oh, K., 2018. A Semantic Search Model Using Word Embedding, POS Tagging, and Named Entity Recognition. *2018 International Conference on Computational Science and Computational Intelligence (CSCI)*. IEEE. Available at: <https://doi.org/10.1109/CSCI46756.2018.00231> [Accessed 19 November 2024].
 19. Uzun, E., Yerlikaya, T. and Kırat, O., 2018. *Comparison of Python libraries used for web data extraction*. In: *International Scientific Conference on Engineering, Technologies and Systems TECHSYS 2018*, Technical University of Sofia, Plovdiv branch, 17–19 May 2018. Plovdiv, Bulgaria: Technical University – Sofia, Plovdiv branch.
 20. Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L. and Stoyanov, V., 2020. *Unsupervised cross-lingual representation learning at scale*. *arXiv preprint arXiv:1911.02116*. Available at: <https://arxiv.org/abs/1911.02116>

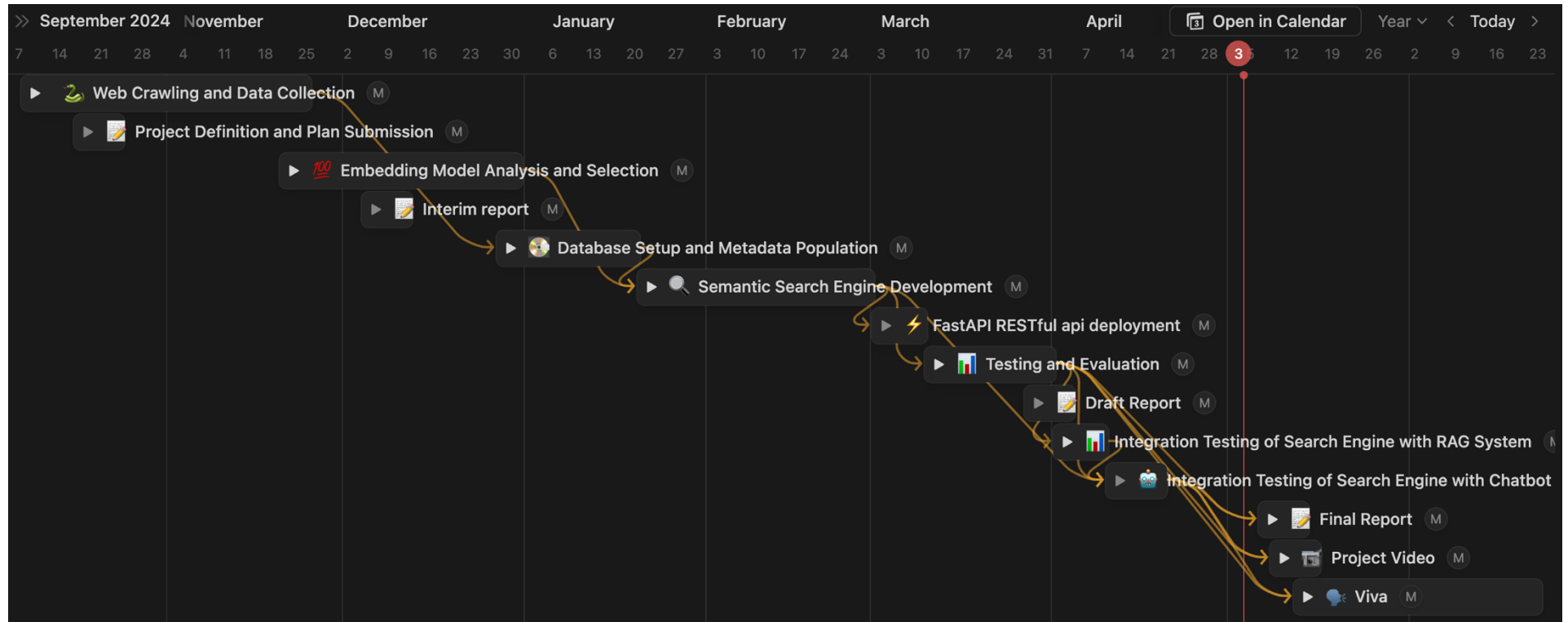
Appendix A – Risk Assessment

<u>ID</u>	<u>Risk</u>	<u>Likelihood</u>	<u>Impact</u>	<u>Actions</u>
1	Difficulty automating data extraction due to non-standard course structures or restrictions.	Medium – Variability in course layouts and dynamic content make automation challenging, though fallback methods are available.	High – Delays in data collection could affect embedding analysis and system development.	Evaluate models for fairness and bias. Apply post-processing corrections where needed.
2	Embedding models fail to meet domain-specific requirements for semantic search.	Medium – Pre-trained models may require fine-tuning for optimal performance in specialised contexts like cybersecurity.	High – Poor embedding quality could reduce the accuracy and relevance of semantic search results, undermining the project's objectives.	Test and fine-tune embedding models using diverse datasets and benchmarks.
3	Challenges in integrating schema-based and vector-based databases.	Low – The hybrid database design is well-researched, but integration complexities can arise.	Medium – Issues during integration could slow development and testing phases.	Optimise database structure, index efficiently, and test scalability early.
4	Time management challenges during complex tasks like web crawling or embedding evaluation.	Medium – High complexity tasks may take longer than anticipated.	Medium – Delays could lead to missing interim or final deadlines.	Set smaller milestones and track weekly progress using Gantt charts, notion or similar tools.
5	Restricted access to IBM SkillsBuild data or embedding model APIs.	Low – Access issues are unlikely but possible.	High – Delayed implementation of the semantic search engine could compromise deliverables.	Secure API access early and prepare backup embedding tools.

6	Data privacy and compliance issues when scraping IBM SkillsBuild content.	Low – Policies are usually clear but must be adhered to strictly.	High – Potential for legal repercussions or project suspension.	Verify IBM's data policies and seek permissions where necessary.
7	Embedding model bias impacting recommendations.	Medium – Bias is common in pre-trained models.	Medium – May lead to misleading course recommendations and affect user satisfaction.	Evaluate models for fairness and bias. Apply post-processing corrections where needed.

Figure 7. Risk assessment table.

Appendix B – Project Plan



Appendix C – Key Implementation code

C.1 – Login Using Credentials from .env File

```
wait_and_perform_action(
    self.driver,
    By.ID,
    'username',
    EC.presence_of_element_located,
    keys=os.getenv("USERNAME"),
    submit=True
)
wait_and_perform_action(
    self.driver,
    By.ID,
    'password',
    EC.presence_of_element_located,
    keys=os.getenv("PASSWORD"),
    submit=True
)
```

C.2 – URL Building Using config.json and .env Files

```
search_sections = self.config.get("search_sections", {})
for section, slug in search_sections.items():
    try:
        keyword = search_category.replace("_", "%20")
        language_filters = self.config.get("language_filters", {})
        language_query = "&".join([f"languages={lang}" for lang in
language_filters.values()])
        base_url = os.getenv("BASE_URL")
        url = f"{base_url}/search/{slug}/q={keyword}&{language_query}"
```

C.3 - Tab Switching Context Manager (__open_new_tab)

```
@contextmanager
def __open_new_tab(self, url: str):
    """
    Context manager to open a new tab, switch to it, and ensure it is
    closed.

    Args:
        url (str): The URL to open in the new tab.
    """
    original_handle = self.driver.current_window_handle
    self.driver.execute_script("window.open(arguments[0]);", url)
    new_handle = self.driver.window_handles[-1]
    self.driver.switch_to.window(new_handle)
    try:
        yield
    finally:
        self.driver.close()
        self.driver.switch_to.window(original_handle)
```

C.4 - Course Page Parser (parse_course_page)

```
def __scrape_course_page(self, link: str, category: str) -> dict[str,
Any]:
    """
    Scrapes a single course page.
```



```

Args:
    link (str): URL of the course page.

Returns:
    dict: Extracted course data.
    :param category:
    """
    with self.__open_new_tab(link):
        try:
            wait_for_element(
                self.driver,
                By.CSS_SELECTOR,
                'div[class*="FullPageDescription_wrapper__CEPjU"] >
div',
                EC.presence_of_element_located
            )
            wait_for_element(
                self.driver,
                By.CSS_SELECTOR,
                '[class^="TagLabel_labelContainer__"] > span',
                EC.presence_of_all_elements_located,
                timeout = int(os.getenv("SCRAPE_DELAY", "2"))
            )
            html_source = self.driver.page_source
            course_data = parse_course_page(
                html_source, link, category
            )
            return course_data if course_data is not None else {}
        except Exception as e:
            logging.error(f"Error scraping course page {link}: {e}")
            return {}

```

C.5 - HTML and Boilerplate Cleaner (html_cleaner.py)

```

class DescriptionCleaner:
    """
    Clean an HTML course-description:
    - strip unwanted tags
    - decode & normalise text
    - remove URLs, numeric garbage, durations & boilerplate
    - extract language codes
    """

    def __init__(self, config: dict[str, Any] | None = None) -> None:
        if config is None:
            config = load_config()
        phrases = load_boilerplate_phrases()

        self.boilerplate_pattern =
self._compile_phrases_pattern(phrases)
        self.duration_pattern = re.compile(
            r"(?:duration|expected duration)\s*:\s*"
            r"(complete the activities.*?learning credit!|"
            r"(?:this course will take you about\s*)?\d+\s*"
            r"(?:hours?|hrs?|h|minutes?|mins?|m) "
            r"(?:,\s*\d+\s*(?:minutes?|mins?|m))?)",
            flags=re.IGNORECASE,
        )

        self.language_span_pattern = re.compile(
            r"(?:languages\s*:\s*[available in|here\s*:"
            r"a version of this course is available in)\s*"
            r"((?:[^\s;\n]|[-\s\ ])+)",

```

```

        flags=re.IGNORECASE,
    )
    self.learners_pattern = re.compile(
        r"(?:\b[a-
z]+\s+)?learners?\s+can\s+complete\s+the\s+learning"
        r"(?:\s+here:)?",
        flags=re.IGNORECASE,
    )

    @staticmethod
    def _compile_phrases_pattern(phrases: list[str]) -> re.Pattern:
        """
        Build a single regex that matches any of the boilerplate
        phrases,
        allowing for trailing punctuation or ellipses.
        """
        escaped = (re.escape(p) for p in phrases)
        pattern = r"|" .join(rf"\s*{p}[\s\.,;:!\u2026\u3002]*" for p
in escaped)
        return re.compile(pattern, flags=re.IGNORECASE)

    @staticmethod
    def _extract_languages_from_soup(soup: BeautifulSoup) ->
list[str]:
        langs = set()
        for tag in soup.find_all(attrs={"lang": True}):
            val = tag.get("lang", "").strip()
            if not val:
                continue
            try:
                lc = langcodes.get(val)
                langs.add(lc.language.lower() if lc and lc.language
else val.lower())
            except Exception:
                langs.add(val.lower())
        return list(langs)

    @staticmethod
    def _extract_iso_languages(text: str) -> tuple[str, list[str]]:
        """
        Pull out any named languages (e.g. "English, Français") and
        return
        (text_without_those_mentions, [iso_codes...])
        """
        candidates = [
            c.strip(" .,:;,"")
            for c in re.split(r"(\band\b)", text)
            if c.strip()
        ]
        iso_codes = set()
        modified = text
        for cand in candidates:
            try:
                lc = langcodes.find(cand)
                if lc and lc.language:
                    iso_codes.add(lc.language.lower())
                    # remove that chunk from the text
                    modified = re.sub(
                        rf"\b{re.escape(cand)}\b",
                        "",
                        modified,
                        flags=re.IGNORECASE,
                    )

```

```

        except Exception:
            continue

    return re.sub(r"\s+", " ", modified).strip(), list(iso_codes)

def clean(self, raw_html: str) -> tuple[str, list[str]]:
    """
    Returns a tuple (clean_text, [language_codes...]).
    """
    if not isinstance(raw_html, str):
        return raw_html, []

    soup = BeautifulSoup(raw_html, "html.parser")
    for tag in soup(["script", "style", "header", "footer", "nav",
"noscript"]):
        tag.decompose()

    html_langs = self._extract_languages_from_soup(soup)

    text = soup.get_text()
    text = html.unescape(text).lower()

    text, text_langs = self._extract_iso_languages(text)

    try:
        text = codecs.decode(text, "unicode_escape")
    except Exception:
        pass

    text = unicodedata.normalize("NFKC", text)
    text = ftty.fix_text(text)
    text = ftty.fix_encoding(text)
    text = re.sub(r"[\u2020\u0304\u00a0]+", " ", text)

    text = re.sub(r"http\S+|www\.\S+", "", text)
    text = re.sub(r"\d{5,}", "", text)

    for pattern in (
        self.language_span_pattern,
        self.duration_pattern,
        self.boilerplate_pattern,
        self.learners_pattern,
    ):
        text = pattern.sub(" ", text)

    langs = set(html_langs) | set(text_langs)
    if not langs:
        langs.add("en")

    text = re.sub(r"([, . ! ? ; : ])\1+", r"\1", text)
    text = re.sub(r"\s+", " ", text).strip()

    return text, list(langs)

```

C.6 - Metadata Extraction Utilities (metadata_extractor.py)

```

def extract_learner_count(text) -> int | None:
    """
    Extracts the number of learners from a string like:
    '229,993 learners have completed this activity in the past 12
    months'

    Returns:
    int: The extracted number of learners, or None if not found.

```

```

"""
if not isinstance(text, str) or pd.isna(text) or not text.strip():
    return None

match = re.search(r"\b(\d{1,10})\b", text)
return int(match.group(1)) if match else None

def extract_star_rating(text) -> float | None:
    """
    Extracts the average star rating from a string like:
    'Average rating of 4.5 stars by 4267 learners in the past 12
    months.'

    Returns:
        float: The extracted rating (e.g. 4.5), or None if not found.
    """
    if not isinstance(text, str) or pd.isna(text) or not text.strip():
        return None

    match = re.search(r"\b(\d+(\.\d+)?)\s*stars?\b", text,
re.IGNORECASE)
    return float(match.group(1)) if match else None

```

C.7 - Combined Text Construction (build_combined_text)

```

def build_combined_text(row: dict) -> str:
    """
    Constructs a labelled, combined text string from title,
    description, and tags.
    Each non-empty field is prefixed with its label and separated by a
    sentence boundary.
    """
    parts = []

    title = row.get("title", "").strip()
    if title:
        parts.append(f"title: {title}")

    description = row.get("description", "").strip()
    if description:
        parts.append(f"description: {description}")

    tags_list = row.get("tags", [])
    if tags_list:
        tags = ", ".join(tag.strip() for tag in tags_list if
tag.strip())
        if tags:
            parts.append(f"tags: {tags}")

    if not parts:
        return ""

    combined = ". ".join(parts)
    if not combined.endswith("."):
        combined += "."

    return combined

```

C.8 - Embedding Computation (compute_embeddings)

```

def compute_embeddings(df: pd.DataFrame, model: SentenceTransformer) -
> pd.DataFrame:

```

```

    """
    Computes the combined text and its corresponding embedding vector
    for each row.
    """
    df["embedding_input_combined"] = df.apply(build_combined_text,
axis=1)
    df["embedding_vector"] = model.encode(
        df["embedding_input_combined"].tolist(),
show_progress_bar=True
    ).tolist()
    return df

```

C.9 - PostgreSQL Table Schema

```

CREATE_TABLE_QUERY = """
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS courses (
    course_id TEXT PRIMARY KEY,
    course_url TEXT NOT NULL UNIQUE,
    category TEXT,
    type TEXT,
    title TEXT,
    duration INTEGER,
    learners_amount INTEGER,
    star_rating REAL,
    star_num_ratings INTEGER,
    description TEXT,
    tags TEXT[],
    title_raw TEXT,
    description_raw TEXT,
    languages TEXT[],
    tags_raw TEXT[],
    embedding_input_combined TEXT,
    embedding_vector VECTOR(384)
);
"""

```

C.10 - Database Insertion Logic (INSERT_QUERY and Upsert)

```

INSERT_QUERY = text("""
INSERT INTO courses (
    course_id, course_url, category, type, title, duration,
    learners_amount, star_rating, star_num_ratings,
    description, tags, title_raw, description_raw, languages,
    tags_raw, embedding_input_combined, embedding_vector
) VALUES (
    :course_id, :course_url, :category, :type, :title, :duration,
:learners_amount, :star_rating, :star_num_ratings,
    :description, :tags, :title_raw, :description_raw, :languages,
:tags_raw, :embedding_input_combined, :embedding_vector
)
ON CONFLICT (course_id)
DO UPDATE SET
    course_url = EXCLUDED.course_url,
    category = EXCLUDED.category,
    type = EXCLUDED.type,
    title = EXCLUDED.title,
    duration = EXCLUDED.duration,
    learners_amount = EXCLUDED.learners_amount,
    star_rating = EXCLUDED.star_rating,
    star_num_ratings = EXCLUDED.star_num_ratings,
    description = EXCLUDED.description,

```

```

tags = EXCLUDED.tags,
title_raw = EXCLUDED.title_raw,
description_raw = EXCLUDED.description_raw,
languages = EXCLUDED.languages,
tags_raw = EXCLUDED.tags_raw,
embedding_input_combined = EXCLUDED.embedding_input_combined,
embedding_vector = EXCLUDED.embedding_vector;
""")

```

C.11 - Query Vector Construction and Filter Handling

```

query_vector = model.encode([query])[0].tolist()
query_vector_str = str(query_vector)

filters = filters or {}
clauses = []
params: dict[str, Any] = {
    "query_vector": query_vector_str,
    "threshold": threshold,
    "limit": limit
}

for col, val in filters.items():
    if isinstance(val, dict):
        if "min" in val and val["min"] is not None:
            clauses.append(f"{col} >= :{col}_min")
            params[f"{col}_min"] = val["min"]
        if "max" in val and val["max"] is not None:
            clauses.append(f"{col} <= :{col}_max")
            params[f"{col}_max"] = val["max"]
    elif isinstance(val, (list, tuple)):
        clauses.append(f"{col} = ANY(:{col})")
        params[col] = val
    else:
        clauses.append(f"{col} = :{col}")
        params[col] = val

extra_where = ""
if clauses:
    extra_where = "\n                AND " + "\n                AND "
".join(clauses)

```

C.12 - Semantic SQL Query Template (pgvector Retrieval)

```

SEMANTIC_RETRIEVAL_QUERY = text(f"""
WITH matches AS (
    SELECT
        course_id,
        course_url,
        title,
        description,
        embedding_input_combined,
        embedding_vector <=> :query_vector AS distance
    FROM courses
    WHERE embedding_vector <=> :query_vector < :threshold
        {extra_where}
    ORDER BY distance
    LIMIT :limit
)
SELECT * FROM matches
""")

```

C.13 - API Integration: Semantic Search Endpoint

```

@app.post(
    "/api/v1/courses/search/semantic",
    response_model=list[CourseOut],
    summary="Semantic search over courses",
    tags=["courses"],
    response_model_exclude_none=True
)
async def semantic_search_endpoint(payload: SearchRequest):
    raw_filters = {
        k: v
        for k, v in (payload.filters or
Filters()).model_dump().items()
        if v is not None
    }
    results = semantic_search(
        query=payload.query,
        model=resources["embedding_model"],
        engine=resources["db_engine"],
        threshold=payload.threshold,
        limit=payload.limit,
        filters=raw_filters
    )
    return results

```

C.14 - Data Models for Semantic Search

```

class RangeFilter(SQLModel):
    min: int | None = Field(None, ge=0, description="Lower bound
(inclusive)")
    max: int | None = Field(None, ge=0, description="Upper bound
(inclusive)")

class Filters(SQLModel):
    star_rating: RangeFilter | None = Field(None, description="Range
of star ratings")
    learners_amount: RangeFilter | None = Field(None,
description="Range of learners amounts")
    duration: RangeFilter | None = Field(None, description="Range of
durations in minutes")
    category: list[str] | None = Field(
        None,
        min_items=1,
        description="List of course categories (e.g.
['cloud_computing', 'cs'])"
    )

class SearchRequest(SQLModel):
    query: str = Field(..., description="Natural-language search
string")
    threshold: float = Field(
        0.5, ge=0.0, le=1.0,
        description="Maximum pgvector distance"
    )
    limit: int = Field(5, ge=1, description="Maximum number of
results")
    filters: Filters | None = Field(None, description="Filters for
specific course attributes")

class CourseOut(SQLModel):
    course_id: str = Field(..., description="Primary key: unique

```

```

course identifier")
    course_url: str = Field(None, description="URL to the course
page")
    category: str | None = Field(None, description="High-level course
category")
    type: str | None = Field(None, description="Course format or
type")
    title: str | None = Field(None, description="Course title")
    duration: int | None = Field(None, description="Duration in
minutes")
    learners_amount: int | None = Field(None, description="Number of
enrolled learners")
    star_rating: float | None = Field(None, description="Average star
rating")
    star_num_ratings: int | None = Field(None, description="Total
number of ratings")
    description: str | None = Field(None, description="Short course
description")
    tags: list[str] | None = Field(None, description="Normalized tags
array")
    languages: list[str] | None = Field(None, description="Languages
in which course is offered")
    embedding_input_combined: str | None = Field(
        None,
        description="Text that was fed into the embedding model"
    )
    embedding_vector: list[float] | None = Field(
        None,
        description="384-dimensional embedding vector (pgvector)",
        exclude=True
    )

class Config:
    orm_mode = True

```


Appendix D - Evaluation Code and Metrics

D.1 - SQL Query for TF-IDF Keyword Search

```

TF_IDF_SEARCH_QUERY = text("""
WITH search_vectors AS (
    SELECT
        course_id,
        title,
        course_url,
        description,
        embedding_input_combined,
        setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
        setweight(to_tsvector('english', coalesce(description, '')),
'B') ||
        setweight(to_tsvector('english',
array_to_string(coalesce(tags, ARRAY[]::TEXT[]), ' '), 'C') AS
document
    FROM courses
)
SELECT
    course_id,
    title,
    course_url,
    description,
    embedding_input_combined,
    ts_rank_cd(document, plainto_tsquery('english', :query_text)) AS
rank
FROM search_vectors
WHERE document @@ plainto_tsquery('english', :query_text)
    AND ts_rank_cd(document, plainto_tsquery('english', :query_text)) >
:threshold
ORDER BY rank DESC
LIMIT :limit
""")

```

D.2 - Preprocessing and Query Expansion

```

def preprocess(text: str) -> list[str]:
    """
    Lemmatise, lowercase, remove stopwords & non-alphabetic tokens,
    and keep only NOUN/VERB/ADJ/PROPN.
    """
    doc = _nlp(text.lower())
    return [
        token.lemma_ for token in doc
        if token.is_alpha and not token.is_stop and token.pos_ in
{"NOUN", "VERB", "ADJ", "PROPN"}
    ]

def get_synonyms(term: str, max_synonyms: int = 3) -> set:
    synsets = wn.synsets(term)
    lemmas = [lemma.name().replace('_', ' ') for syn in synsets for
lemma in syn.lemmas()]
    unique_lemmas = list(dict.fromkeys(lemmas))
    return set(unique_lemmas[:max_synonyms]) - {term}

def expand_query(terms: list[str]) -> list[str]:
    """

```

```

Expand the original terms set with their WordNet synonyms.
"""
expanded = set(terms)
for term in terms:
    expanded |= get_synonyms(term)
return list(expanded)

```

D.3 - BM25 Scoring Function

```

def bm25_score(
    query_terms: list[str],
    doc_terms: list[str],
    avgdl: float,
    k1: float = 1.2,
    b: float = 0.75,
    N: int = 1,
    df: dict[str, int] = None
) -> float:
    """
    Compute BM25 score for a single document.
    """
    df = df or {}
    freqs = Counter(doc_terms)
    score = 0.0

    for term in query_terms:
        f = freqs.get(term, 0)
        n = df.get(term, 0)
        idf = math.log((N - n + 0.5) / (n + 0.5) + 1)
        numerator = f * (k1 + 1)
        denominator = f + k1 * (1 - b + b * (len(doc_terms) / avgdl))
        score += idf * (numerator / denominator)

    return score

```

D.4 - Full Evaluation Pipeline (BM25 Search)

```

def search_courses_bm25(
    query: str,
    engine: Engine,
    limit: int = 5,
    k1: float = 1.2,
    b: float = 0.75
) -> list[dict[str, Any]]:
    if _courses is None or _nlp is None:
        setup_environment(engine)

    lemmas = preprocess(query)
    if not lemmas:
        return []

    expanded_terms = expand_query(lemmas)

    stmt = select(_courses)
    for term in lemmas:
        stmt =
    stmt.where(_courses.c.embedding_input_combined.ilike(f"%{term}%"))

    with engine.connect() as conn:
        rows = conn.execute(stmt).mappings().all()

    if not rows:
        return []

```

```

documents = [
    preprocess(row['embedding_input_combined'] or "")
    for row in rows
]
N = len(documents)
avgdl = sum(len(doc) for doc in documents) / N if N > 0 else 1.0

df = Counter()
for doc in documents:
    for term in set(doc):
        df[term] += 1

scored = []
for idx, row in enumerate(rows):
    score = bm25_score(expanded_terms, documents[idx], avgdl, k1,
b, N=N, df=df)
    result = dict(row)
    result['bm25_score'] = score
    scored.append(result)

scored.sort(key=lambda x: x['bm25_score'], reverse=True)
return scored[:limit]

```

D.5 - Core Evaluation Metrics

```

def precision_at_k(actual: list[str], predicted: list[str], k: int) ->
float:
    """
    Precision@k = (# of relevant items in top-k) / k
    """
    if k <= 0:
        return 0.0
    act_set = set(actual)
    topk = predicted[:k]
    return len(act_set & set(topk)) / k

def recall_at_k(actual: list[str], predicted: list[str], k: int) ->
float:
    if not actual:
        return 0.0
    act_set = set(actual)
    return len(act_set & set(predicted[:k])) / len(act_set)

def reciprocal_rank(actual: list[str], predicted: list[str]) -> float:
    for i, p in enumerate(predicted, start=1):
        if p in actual:
            return 1.0 / i
    return 0.0

def average_precision(actual: list[str], predicted: list[str], k: int)
-> float:
    if not actual:
        return 0.0
    act_set = set(actual)
    hits = 0
    score = 0.0
    for i, p in enumerate(predicted[:k], start=1):
        if p in act_set:
            hits += 1

```

```

        score += hits / i
    return score / len(act_set)

def ndcg_at_k(actual_items: list[dict], predicted: list[str], k: int)
-> float:
    def dcg(rels: list[float]) -> float:
        return sum(r / math.log2(idx + 1) for idx, r in
enumerate(rels, start=1))

    rel_map = {item["course_id"]: item.get("relevance", 0) for item in
actual_items}
    preds_rels = [rel_map.get(pid, 0) for pid in predicted[:k]]
    ideal_rels = sorted(rel_map.values(), reverse=True)[:k]

    max_dcg = dcg(ideal_rels)
    return dcg(preds_rels) / max_dcg if max_dcg > 0 else 0.0

```

D.6 - Evaluation Orchestrator

```

def evaluate(
    predictions: dict[str, list[str]],
    test_dataset: dict[str, list[dict]],
    k: int,
) -> dict[str, float]:
    """
    Compute average Precision@k, Recall@k, MRR, MAP@k, nDCG@k over all
    queries.
    """
    precisions, recs, rrs, aps, ndcgs = [], [], [], [], []
    for query, actual_items in test_dataset.items():
        actual_ids = [
            itm["course_id"]
            for itm in actual_items
            if itm.get("relevance", 1) > 0
        ]
        preds = predictions.get(query, [])
        precisions.append(precision_at_k(actual_ids, preds, k))
        recs.append(recall_at_k(actual_ids, preds, k))
        rrs.append(reciprocal_rank(actual_ids, preds))
        aps.append(average_precision(actual_ids, preds, k))
        ndcgs.append(ndcg_at_k(actual_items, preds, k))

    n = max(len(test_dataset), 1)
    return {
        f"Precision@{k}": round(sum(precisions) / n, 4),
        f"Recall@{k}": round(sum(recs) / n, 4),
        "MRR": round(sum(rrs) / n, 4),
        f"MAP@{k}": round(sum(aps) / n, 4),
        f"nDCG@{k}": round(sum(ndcgs) / n, 4),
    }

```

D.7 - Visualisation: Metrics vs. k

```

def plot_metrics_vs_k(
    preds: dict[str, dict[str, list[str]]],
    test_ds: dict[str, list[dict]],
    ks: list[int],
) -> None:
    fig, ax = plt.subplots(figsize=(8, 5))

    for sys, pm in preds.items():
        recs = [evaluate(pm, test_ds, k)[f"Recall@{k}"] for k in

```

```

ks]
    preds= [evaluate(pm, test_ds, k)[f"Precision@{k}"]] for k in
ks]
    maps = [evaluate(pm, test_ds, k)[f"MAP@{k}"]] for k in
ks]
    nds = [evaluate(pm, test_ds, k)[f"nDCG@{k}"]] for k in
ks]

    ax.plot(ks, preds, marker="x", linestyle="-.", label=f"{sys}
Precision")
    ax.plot(ks, recs, marker="o", linestyle="-", label=f"{sys}
Recall")
    ax.plot(ks, maps, marker="s", linestyle="--", label=f"{sys}
MAP")
    ax.plot(ks, nds, marker="^", linestyle=":", label=f"{sys}
nDCG")

    ax.set_xlabel("k")
    ax.set_ylabel("Score")
    ax.set_title("Metrics vs k")
    ax.grid(linestyle="--", alpha=0.5)
    ax.legend(fontsize="small", ncol=2, loc="upper left",
bbox_to_anchor=(1, 1))
    plt.tight_layout()
    plt.show()

```

D.8 - Performance Logging (Latency & Coverage)

```

def compute_coverage(preds: dict[str, list[str]]) -> float:
    return sum(bool(v) for v in preds.values()) / len(preds)

def compute_average_latency(times: list[float]) -> float:
    return sum(times) / len(times)

```

Appendix E - Example Evaluation Queries

1. **"build and deploy a CNN image classifier project"**
(*Skill-building, Machine Learning*)
1. **"learn to design responsive web layouts with CSS and Flexbox"**
(*Web Development, Practical UI skills*)
2. **"beginner artificial intelligence course for non-programmers"**
(*Beginner orientation, AI*)
3. **"foundations of reinforcement learning for beginners"**
(*Introductory Machine Learning*)
4. **"beginner friendly course on version control with Git and GitHub"**
(*Foundational DevOps skills*)
5. **"secure web development best practices course"**
(*Cybersecurity in Development*)
6. **"project-based learning: build a RESTful API in Node.js"**
(*Hands-on Web Backend Development*)
7. **"introductory machine learning with Python tutorial"**
(*Technical, beginner-friendly, ML*)